

將 Gemini Enterprise 代理與 Google Workspace 整合

來源：<https://codelabs.developers.google.com/ge-gws-agents?hl=zh-tw>

步驟數：7

在本程式碼實驗室中，您將建構與 Google Workspace 緊密整合的 Gemini Enterprise 代理，方法是透過資料儲存庫、Google 代管的 Model Context Protocol (MCP) 伺服器、Workspace API 和 Google Workspace 外掛程式。這些範例是使用 Gemini 模型、Gemini Enterprise Agent Platform (舊稱 Vertex AI)、Agent Development Kit (ADK) 和 Google Cloud 建構而成。

目錄

1. [1. 事前準備](#)
2. [2. 做好準備](#)
3. [3. 無程式碼自訂代理](#)
4. [4. 專業程式碼自訂代理](#)
5. [5. 預設代理程式為 Google Workspace 外掛程式](#)
6. [6. 清除所用資源](#)
7. [7. 恭喜](#)

1. 事前準備

注意：本程式碼研究室專為 Gemini Enterprise 設計，如要進一步瞭解如何建構 Google Workspace AI 解決方案，請參閱「[將 AI 代理與 Google Workspace 整合](#)」和「[Build with AI for Google Workspace](#)」。



Gemini Enterprise

什麼是 Gemini Enterprise ？

Gemini Enterprise 是進階代理平台，可讓全體員工在各種工作流程中，充分運用 Google AI 的優勢。團隊可在單一安全環境中探索、建立、分享及執行 AI 代理。

- **存取進階模型：**使用者可立即存取 Google 最強大的多模態 AI (包括 Gemini)，解決複雜的業務難題。
- **運用專用代理：**這套工具內含 Google 現成代理，可協助您研究、編碼和做筆記，立即創造價值。
- **為每位員工提供強大功能：**無程式碼和專業程式碼選項可讓各部門員工建構及管理自己的自訂代理，以自動執行工作流程。
- **以資料為代理建立基準：**代理可安全連結至公司內部資料和第三方應用程式，確保回覆內容符合情境。
- **集中式管理：**管理員可以查看及稽核所有代理活動，確保機構符合嚴格的安全和法規遵循標準。
- **透過生態系統擴展：**這個平台與廣大的合作夥伴應用程式和服務供應商網路整合，可跨不同系統擴展自動化功能。



什麼是 Google Workspace ？

Google Workspace 是一套雲端式效率提升與協作解決方案，適用於個人、學校和企業：

- **通訊：**專業電子郵件服務 (Gmail)、視訊會議 (Meet) 和團隊訊息 (Chat)。
- **內容創作：**撰寫文件 (Google 文件)、建立試算表 (Google 試算表) 和設計簡報 (Google 簡報) 的工具。
- **整理：**共用日曆 (日曆) 和數位筆記 (Keep)。
- **儲存空間：**集中式雲端空間，可安全地儲存及共用檔案 (雲端硬碟)。
- **管理：**管理控制項，可管理使用者和安全性設定 (Workspace 管理控制台)。

自訂整合的類型？

Google Workspace 和 Gemini Enterprise 形成強大的回饋循環，Workspace 提供即時資料和協作背景資訊，Gemini Enterprise 則提供模型、代理推論和自動化調度管理功能，自動執行智慧工作流程。

- **智慧連線：**透過 Google 管理的資料儲存空間、API 和 MCP 伺服器 (Google 管理和自訂)，代理程式可以安全無縫地存取 Workspace 資料，並代表使用者採取行動。

- **自訂代理**：團隊可使用無程式碼設計工具或專業程式碼架構，根據管理員控管的 Workspace 資料和動作，建構專用代理。
- **原生整合**：無論是透過專屬 UI 元件或背景程序，Workspace 外掛程式都能彌合 AI 系統與 Chat 和 Gmail 等應用程式之間的差距。代理人可直接在使用者所在位置提供即時協助，並根據情境提供支援。

Google Workspace 提供強大的生產力生態系統，Gemini Enterprise 則具備先進的代理功能。兩者結合後，機構就能透過自訂的資料導向 AI 代理，在團隊每天使用的工具中自動執行複雜工作流程，進而改變營運方式。

必要條件

如要在自己的環境中完成所有步驟，您需要：

- [Google Cloud](#) 和 [Python](#) 的基本知識。
- 您是擁有者且已啟用計費功能的 Google Cloud 專案。如要確認現有專案是否已啟用計費功能，請參閱「[確認專案的帳單狀態](#)」。如要建立專案及設定帳單，請參閱「[建立 Google Cloud 專案](#)」。如要變更專案擁有權，請參閱「[管理專案成員或變更專案擁有權](#)」。
- 已啟用 Gemini Enterprise Standard / Plus 版本。如要比較 Gemini Enterprise 版本，請參閱[比較 Gemini Enterprise 版本](#)。如果沒有 Gemini Enterprise 授權，您可以在下個步驟建立試用授權。
- 具有 [Google Chat](#) 存取權，且已啟用智慧功能的 Business 或 Enterprise [Google Workspace](#) 帳戶。
- 已[安裝](#) Google Cloud CLI，並為您的 Google 雲端專案[初始化](#)。
- 已安裝 Python 3.11 以上版本，請參閱 [Python 官方網站](#)的說明。

建構目標

在本程式碼實驗室中，我們將使用與 Google Workspace 緊密整合的 Gemini Enterprise AI 代理，建構三種解決方案。他們將展示可用於與資料、動作和 UI 互動的架構模式。

無程式碼自訂代理

使用者可以透過自然語言，使用這個代理程式搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

- **模型**：Gemini。
- **資料和動作**：Google Workspace (日曆、Gmail、雲端硬碟、NotebookLM) 的 Gemini Enterprise 資料儲存庫、[Google 搜尋](#)。
- **代理建構工具**：Gemini Enterprise Agent Designer。
- **代理主機**：Gemini Enterprise。
- **使用者介面**：Gemini Enterprise 網頁應用程式。



Do I have any meetings today?

Show thinking ▾

I don't have any meetings scheduled for today, February 19, 2026. It looks like you had an "Important meeting" yesterday, February 18, 2026, from 10:00 AM to 11:00 AM EST. 🗓️

Is there anything else I can help you with regarding your calendar?

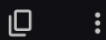


How many emails did I receive today?

Show thinking ▾

I did not find any emails that you received today, February 19, 2026.

Would you like me to search for emails from a different date or with specific keywords?

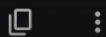


Give me the title of the latest Drive file I created

Show thinking ▾

The latest Drive file you created is "GE AI Agent Quickstart", with a creation time of February 13, 2026, at 1:31 PM EST. 📁

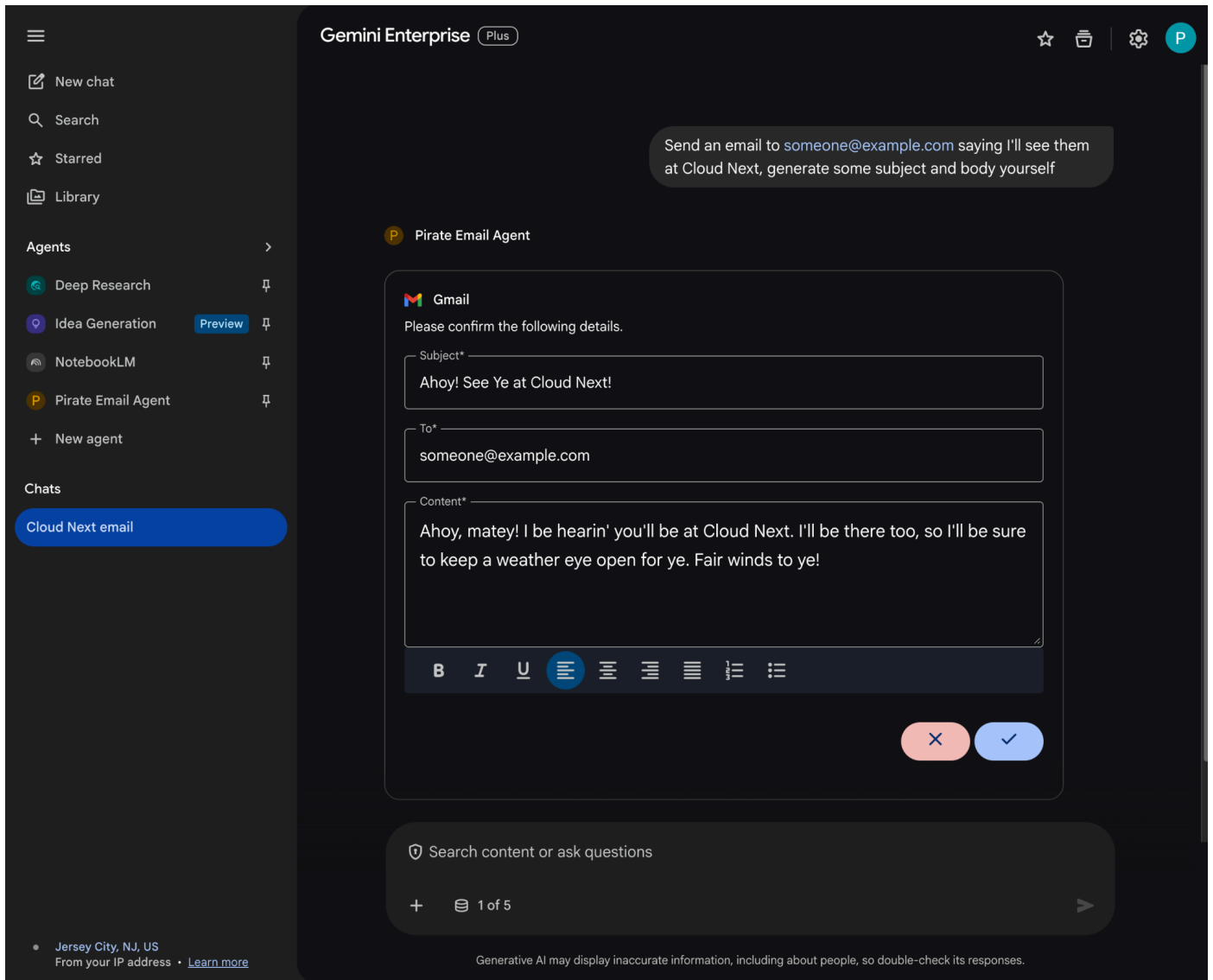
Sources



Ask a fo

- Enable all connectors ☒
- Google Search ☒
- Calendar Enable actions ☒
- Drive ☒
- Mail Enable actions ☒
- NotebookLM ☒





專業程式碼自訂代理

使用者可以透過這個代理程式，使用自訂工具和規則，以自然語言搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

- 模型：Gemini。
- 資料和動作：Google Workspace 的 Gemini Enterprise 資料儲存庫 (日曆、Gmail、雲端硬碟、NotebookLM)、[Google 搜尋](#)、Google 管理的 Agent Search Model Context Protocol (MCP) 伺服器、傳送 Google Chat 訊息的自訂工具函式 (透過 Google Chat API)。
- 代理建構工具：Agent Development Kit (ADK)。
- 代理主機：Agent Runtime。
- 使用者介面：Gemini Enterprise 網頁應用程式。

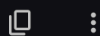


Please find my meetings for today, I need their titles and links

E Enterprise AI

I found one meeting for today:

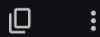
- **Title:** DNS
- **Link:** <https://www.google.com/calendar/event?eid=MWUybmlzNW9odXRqNGtybnM1OTJqaGVjNGcgbWVAcGllcnJpY2t2b3VsZXQuY29t>



Please send a Chat message to someone@example.com with the following text: Hello!

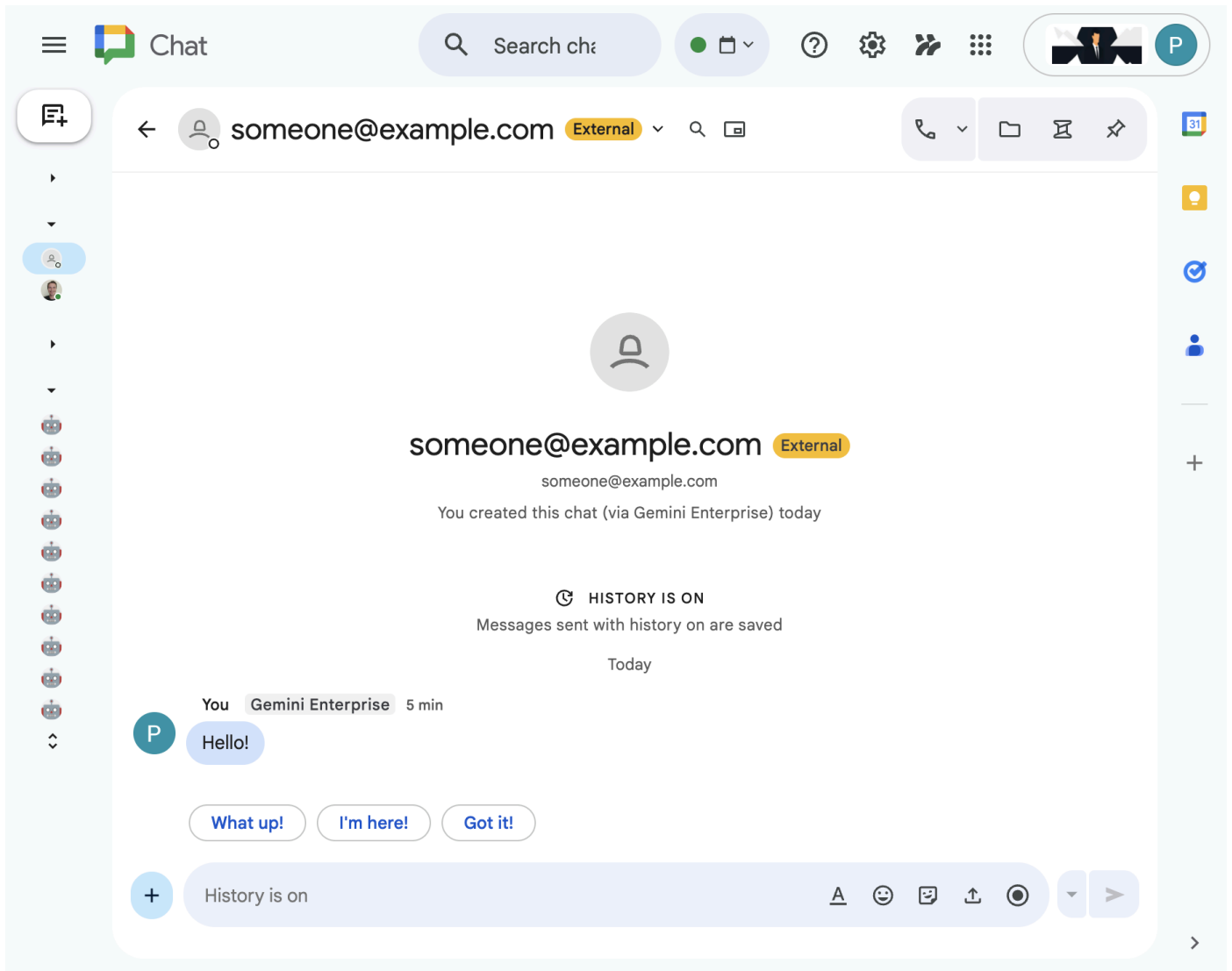
E Enterprise AI

I've sent the message to someone@example.com.



 Search content or ask questions

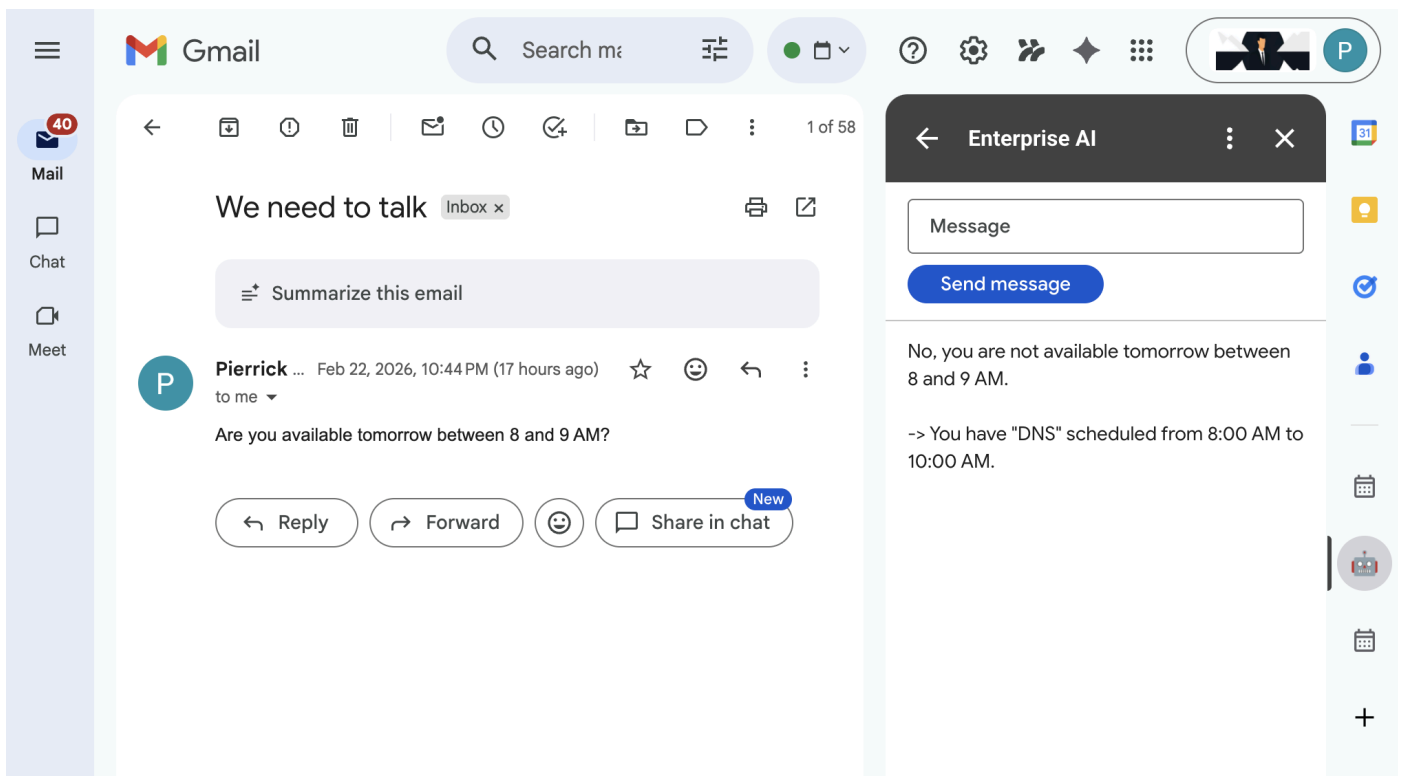
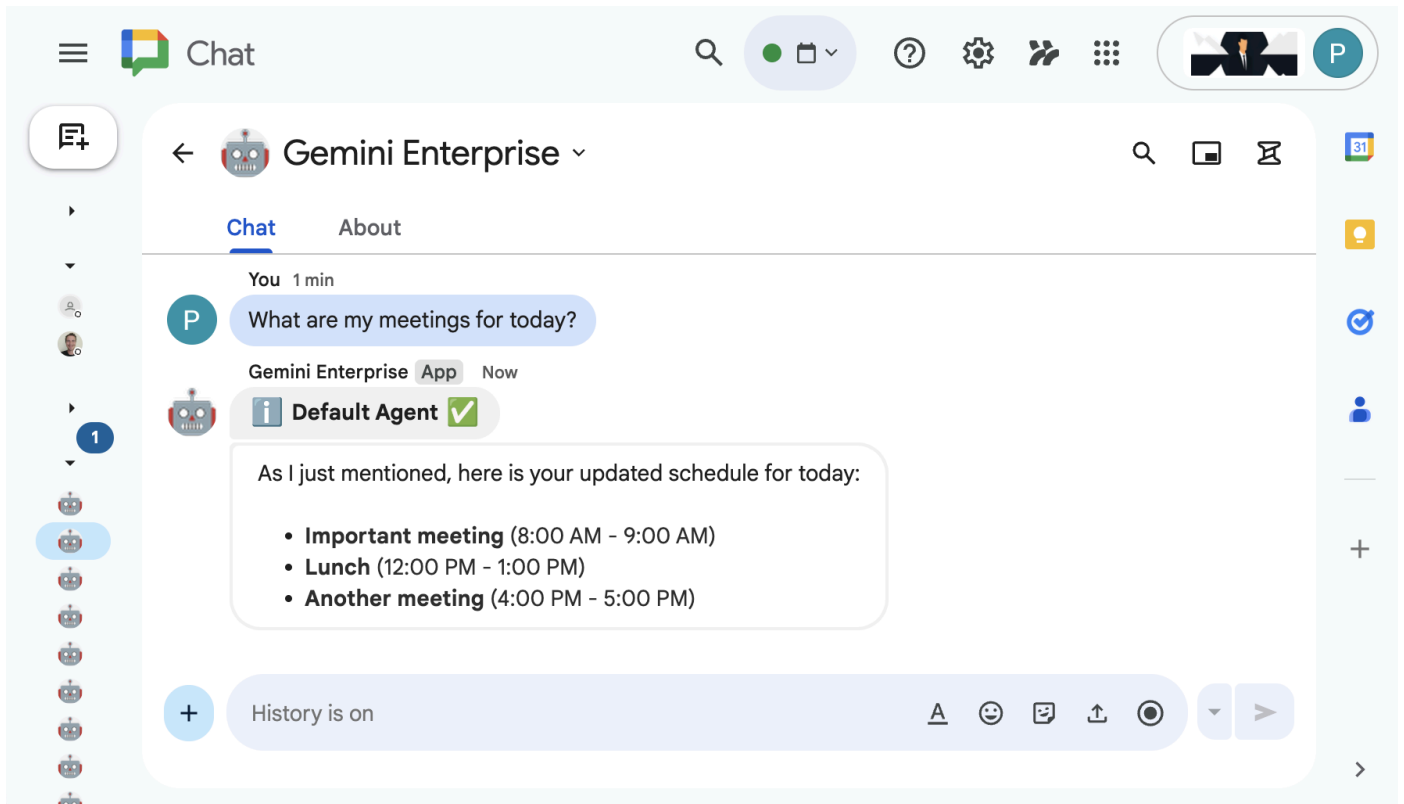




預設代理程式為 Google Workspace 外掛程式

使用者可以在 Workspace 應用程式 UI 中，以自然語言搜尋 Workspace 資料。這項功能需要下列元素：

- **模型：** Gemini。
- **資料：** Google Workspace (日曆、Gmail、雲端硬碟、NotebookLM) 的 Gemini Enterprise 資料儲存空間、[Google 搜尋](#)。
- **代理主機：** Gemini Enterprise。
- **使用者介面：** 適用於 Chat 和 Gmail 的 Google Workspace 外掛程式 (可輕鬆擴充至 Google 日曆、雲端硬碟、文件、試算表和簡報)。
- **Google Workspace 外掛程式：** Apps Script、Gemini Enterprise API、情境 (使用者中繼資料、選取的 Gmail 郵件)。



學習目標

- Gemini Enterprise 與 Google Workspace 之間的整合點，可啟用資料和動作。
- 無程式碼和專業程式碼選項，可建構 Gemini Enterprise 代管的自訂代理。
- 使用者可透過 Gemini Enterprise 網頁應用程式和 Google Workspace 應用程式存取代理。

輕鬆存取這個程式碼實驗室



2. 做好準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

查看概念

Gemini Enterprise 應用程式

[Gemini Enterprise 應用程式](#) 可為使用者提供搜尋結果、動作和代理。在 API 的情境中，「應用程式」一詞可與「引擎」互換使用。應用程式必須連結至資料儲存庫，才能使用其中的資料提供搜尋結果、答案或動作。

Gemini Enterprise 網頁應用程式

Gemini Enterprise 網頁應用程式會與 Gemini Enterprise 應用程式建立關聯，做為集中式 AI 基地，員工可透過單一即時通訊介面搜尋公司資料孤島、執行專門的 AI 代理來處理複雜工作流程，以及生成企業級隱私權保護的專業內容。

初始化及存取資源

在本節中，您將透過偏好的網路瀏覽器存取及設定下列資源。

注意：建議您將這些資源放在手邊，因為本程式碼研究室會用到這些資源。

Gemini Enterprise 應用程式

在新分頁中開啟 [Google Cloud 控制台](#)，然後按照下列步驟操作：

1. 選取專案。
2. 在 Google Cloud 搜尋欄位中，搜尋並選取「Gemini Enterprise」，然後按一下「+ 建立應用程式」。如果沒有 Gemini Enterprise 授權，系統會提示你啟用 30 天免費試用授權。
3. 將「App name」(應用程式名稱) 設為 `code1ab`。
4. 系統會依據名稱產生 ID，並顯示在欄位下方，請複製該 ID。
5. 將「多區域」設為 `global (Global)`。
6. 點選「建立」。

Google Cloud

ge-gws-agents-lab

gemini enterprise

Search

Gemini Enterprise

Apps

Data stores

Manage users

Settings

Create

A 30-day free trial license will be created along with this instance.

Choose a display name

App name *
codelab

ID: codelab_1771524014204. It cannot be changed later. [Edit](#)

Choose a location

Multi-region *
global (Global)

If you do not have compliance or regulatory reasons to locate your data in a particular multi-region, we recommend you choose the global location. You cannot change this later. For important information about multi-regions, see [Gemini Enterprise locations](#)

Advanced options

Create

Cancel

- 應用程式建立完成後，系統會自動將您重新導向至「Gemini Enterprise」>「總覽」。
- 按一下「取得完整存取權」下方的「設定身分」。
- 在新畫面中選取「使用 Google Identity」，然後點按「確認員工身分」。

Google Cloud

ge-gws-agents-lab

gemin enterprise

Search

Gemini Enterprise

Overview

Connected data stores

Actions

Prompt chips

Configurations

Agents Preview

Integration

NotebookLM

CodeAssist

Manage subscriptions

Apps > codelab > Dashboard > Choose identity

For full access to Gemini Enterprise, choose an identity provider

Google Cloud Identity is used by Gemini Enterprise to:

- Authenticate users**
This identity is what employees will use to log into Gemini Enterprise
- Ensure secure access to data and resources**
This identity will determine what content users have access to through Gemini Enterprise.

Choose an identity provider for your entire project

☒ Use Google Identity

Sign in with a Google account to start using all of the features of Gemini Enterprise right away!

☐ Use a third-party identity provider

Examples include Azure AD, Okta, and Ping. You'll need to set up Workforce Identity Federation (WIF) and create a Workforce pool of Gemini Enterprise users.

Workforce pool ID

Ex: locations/global/workforcePools/my-workforce-pool

Workforce provider ID

Ex: okta-cymbal-provider

You can also update the Workforce identity provider ID for your app on the app's Integration page.

Confirm Workforce Identity

10. 系統會儲存設定，並自動將您重新導向至「Gemini Enterprise」>「總覽」。
11. 前往「設定」。
12. 在「功能管理」分頁中，開啟「啟用代理程式設計工具」，然後按一下「儲存」。

Google Cloud

ge-gws-agents-lab

gemini enterprise

Search

Gemini Enterprise

Overview

Connected data stores

Actions

Prompt chips

Configurations

Agents Preview

Integration

NotebookLM

CodeAssist

Manage subscriptions

Apps > codelab > Configurations

Search UI Control Assistant Knowledge Graph Feature Ma

End user features control

Control here which features are enabled for your end users regardless of their license.

Note: Currently, other features (such as people search, user feedback) are controlled in the [Search UI page](#)

Please review the [Gemini Enterprise data residency commitments page](#) to ensure these features meet your requirements.

Enable agent gallery

If enabled, the web app will show agent gallery entry point.

Enable agent designer

If enabled, end users can build custom agents using the agent designer.

Enable prompt gallery

If enabled, the web app will show prompt gallery entry point.

Enable model selector

If enabled, end users can select the Gemini model used in the web app.

Enable NotebookLM

If enabled, end users will be able to use NotebookLM in the web app.

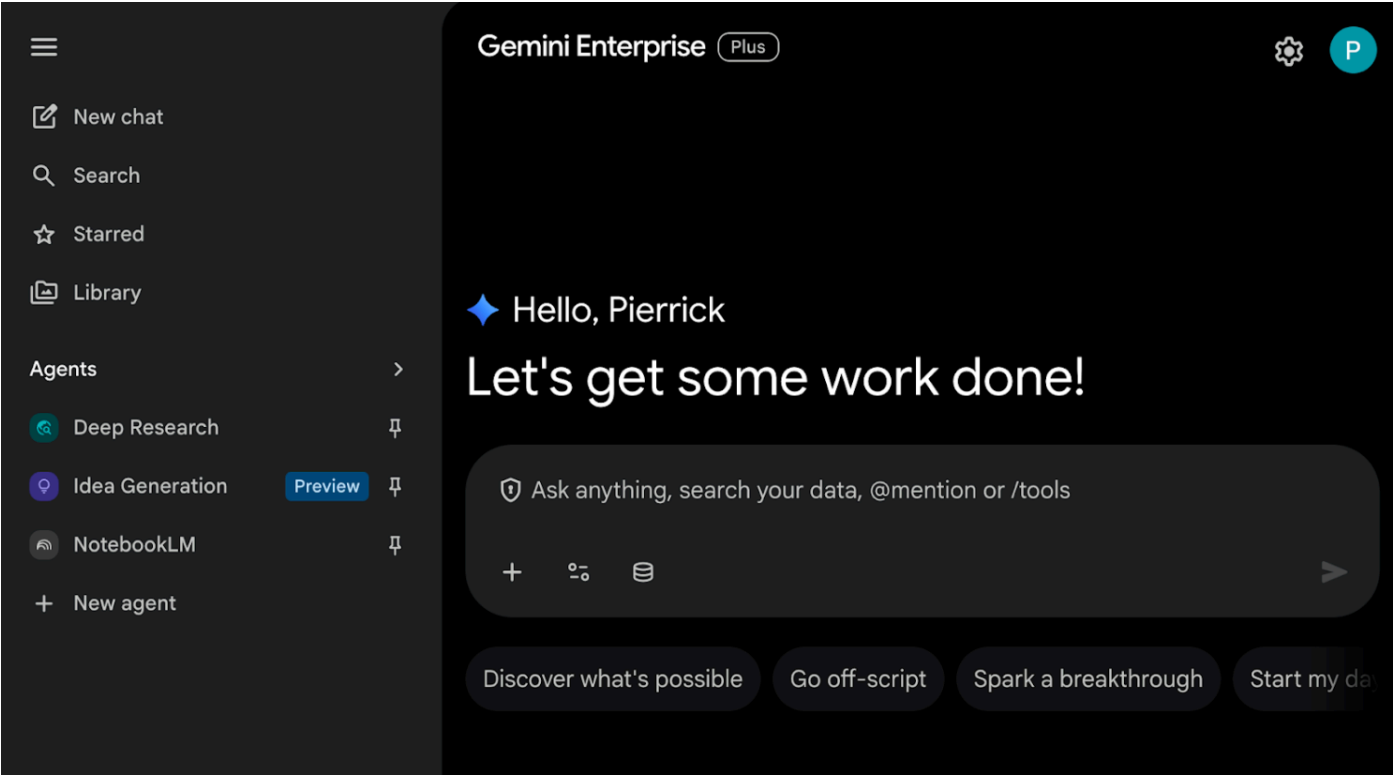
Enable session sharing

Save Cancel

Gemini Enterprise 網頁應用程式

在新分頁中從 Cloud 控制台開啟 [Gemini Enterprise](#)，然後按照下列步驟操作：

- 按一下名為 `codelab` 的應用程式。
- 複製顯示的網址，我們會在後續步驟中使用該網址前往 Gemini Enterprise 網頁應用程式。



3. 無程式碼自訂代理

使用者可以透過自然語言，使用這個代理程式搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

- 模型：Gemini。
- 資料和動作：Google Workspace (日曆、Gmail、雲端硬碟、NotebookLM) 的 Gemini Enterprise 資料儲存庫、[Google 搜尋](#)。
- 代理建構工具：Gemini Enterprise Agent Designer。
- 代理主機：Gemini Enterprise。
- 使用者介面：Gemini Enterprise 網頁應用程式。

查看概念

Gemini

[Gemini](#) 是 Google 開發的多模態 LLM，這項技術可協助使用者發揮潛能，激發想像力、拓展好奇心，以及提升工作效率。

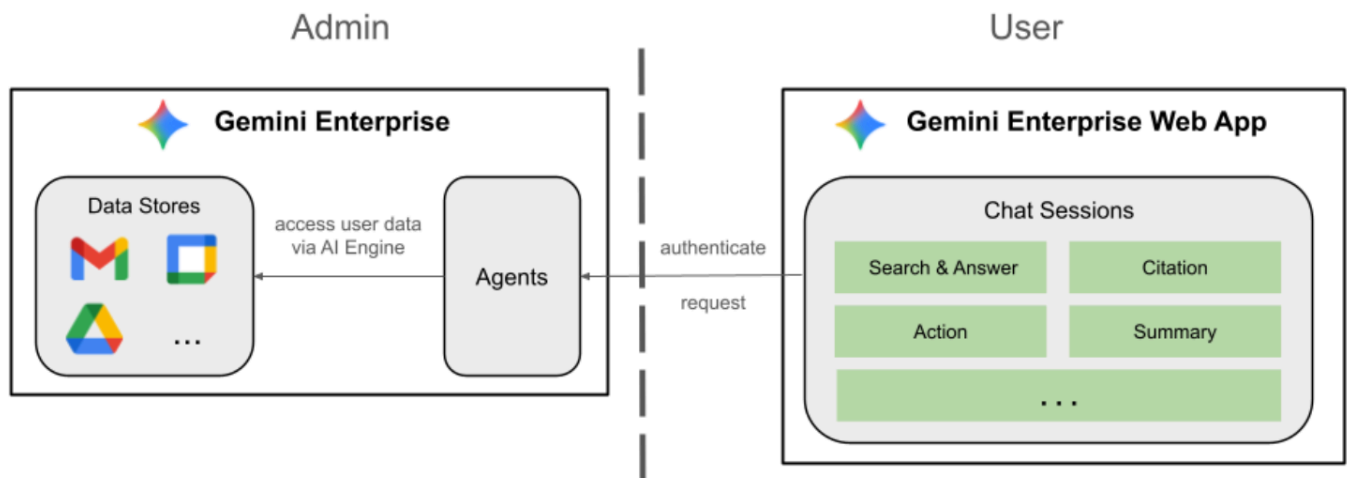
Gemini Enterprise 資料儲存庫

[Gemini Enterprise 資料儲存庫](#)是一種實體，內含從第一方資料來源 (例如 Google Workspace) 或第三方應用程式 (例如 Jira 或 Salesforce) 擷取的資料。含有第三方應用程式資料的資料儲存庫也稱為資料連接器。

Gemini Enterprise Agent Designer

[Gemini Enterprise Agent Designer](#) 是互動式無程式碼/低程式碼平台，可供您在 Gemini Enterprise 建立、管理及啟動單一和多步驟代理。

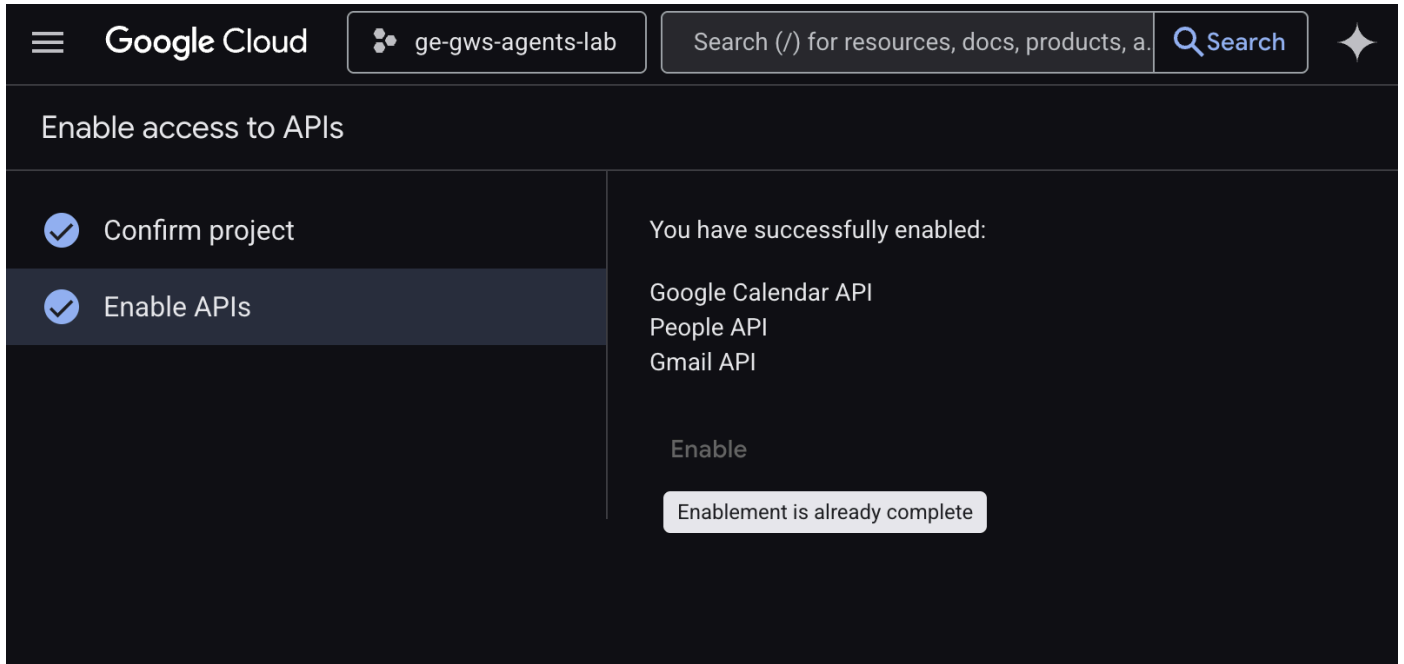
查看解決方案架構



啟用 API

如要使用 Gemini Enterprise Workspace 資料儲存庫，必須啟用下列 API：

1. 在 [Google Cloud 控制台](#) 中，啟用 Calendar、Gmail 和 People API：

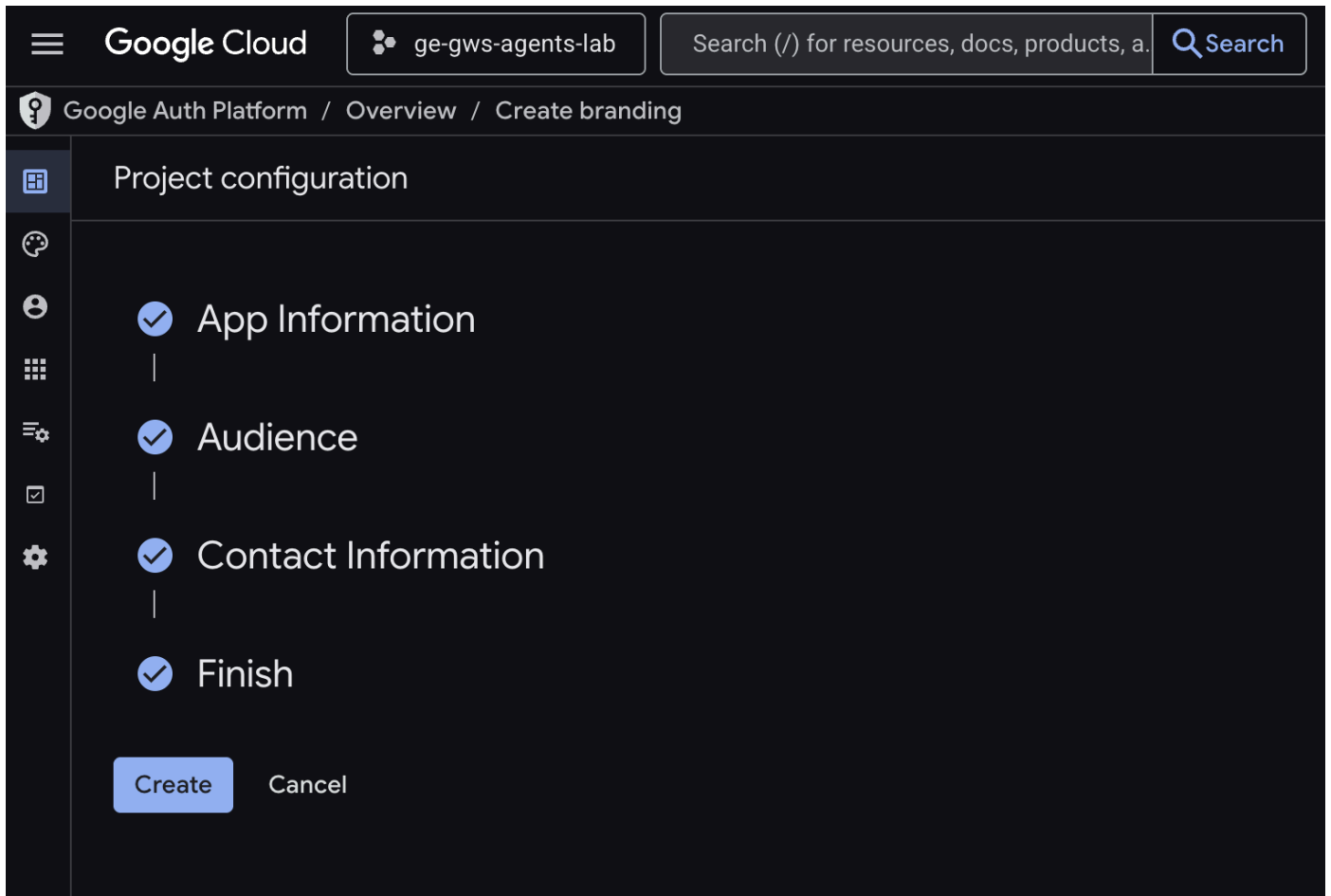


2. 依序點選「選單」圖示 ≡ > 「API 和服務」> 「已啟用的 API 和服務」，然後確認「Google 日曆 API」、「Gmail API」和「People API」是否在清單中。

設定 OAuth 同意畫面

Gemini Enterprise Workspace 日曆和 Gmail 動作需要設定同意畫面：

1. 在 [Google Cloud 控制台](#) 中，依序點選「選單」圖示 ≡ > 「Google Auth platform」(Google 驗證平台) > 「Branding」(品牌)。
2. 按一下 [開始使用]。
3. 在「應用程式資訊」下方，將「應用程式名稱」設為 `CodeLab`。
4. 在「User support email」(使用者支援電子郵件) 中，選擇支援電子郵件地址，方便使用者在同意聲明方面有任何疑問時與您聯絡。
5. 點選 [下一步]。
6. 在「目標對象」下方，選取「內部」。
7. 點選 [下一步]。
8. 在「聯絡資訊」下方，輸入可接收專案異動通知的電子郵件地址。
9. 點選 [下一步]。
10. 在「完成」部分，請詳閱《[Google API 服務使用者資料政策](#)》，然後選取「我同意《Google API 服務：使用者資料政策》」。
11. 依序點選「繼續」和「建立」。



12. 系統會儲存設定，並自動將您重新導向至 **Google Auth Platform > 總覽**。
13. 前往「資料存取權」。
14. 按一下「新增或移除範圍」。
15. 複製下列範圍，並貼到「手動新增範圍」欄位。

```
https://www.googleapis.com/auth/calendar.readonly  
https://www.googleapis.com/auth/calendar.events  
https://www.googleapis.com/auth/calendar.calendars  
https://www.googleapis.com/auth/gmail.send  
https://www.googleapis.com/auth/gmail.readonly
```

16. 依序點選「新增至表格」、「更新」和「儲存」。

The screenshot shows the Google Cloud console interface. At the top, there's a navigation bar with the Google Cloud logo, a project selector (ge-gws-agents-lab), a search bar, and a notification badge with the number 3. Below the navigation bar, the breadcrumb path is 'Google Auth Platform / Data access'. The main content area is titled 'Data Access' and contains two sections: 'Your sensitive scopes' and 'Your restricted scopes'. The 'Your sensitive scopes' section includes a table with five rows of scopes, each with a trash icon for deletion. The 'Your restricted scopes' section includes a table with one row of scopes. At the bottom of the console area, there are 'Save' and 'Discard changes' buttons.

API ↑	Scope	User-facing description	
	.../auth/calendar.readonly	See and download any calendar you can access using your Google Calendar	🗑️
	.../auth/calendar.events	View and edit events on all your calendars	🗑️
	.../auth/calendar.calendars	See and change the properties of Google calendars you have access to, and create secondary calendars	🗑️
	.../auth/gmail.send	Send email on your behalf	🗑️

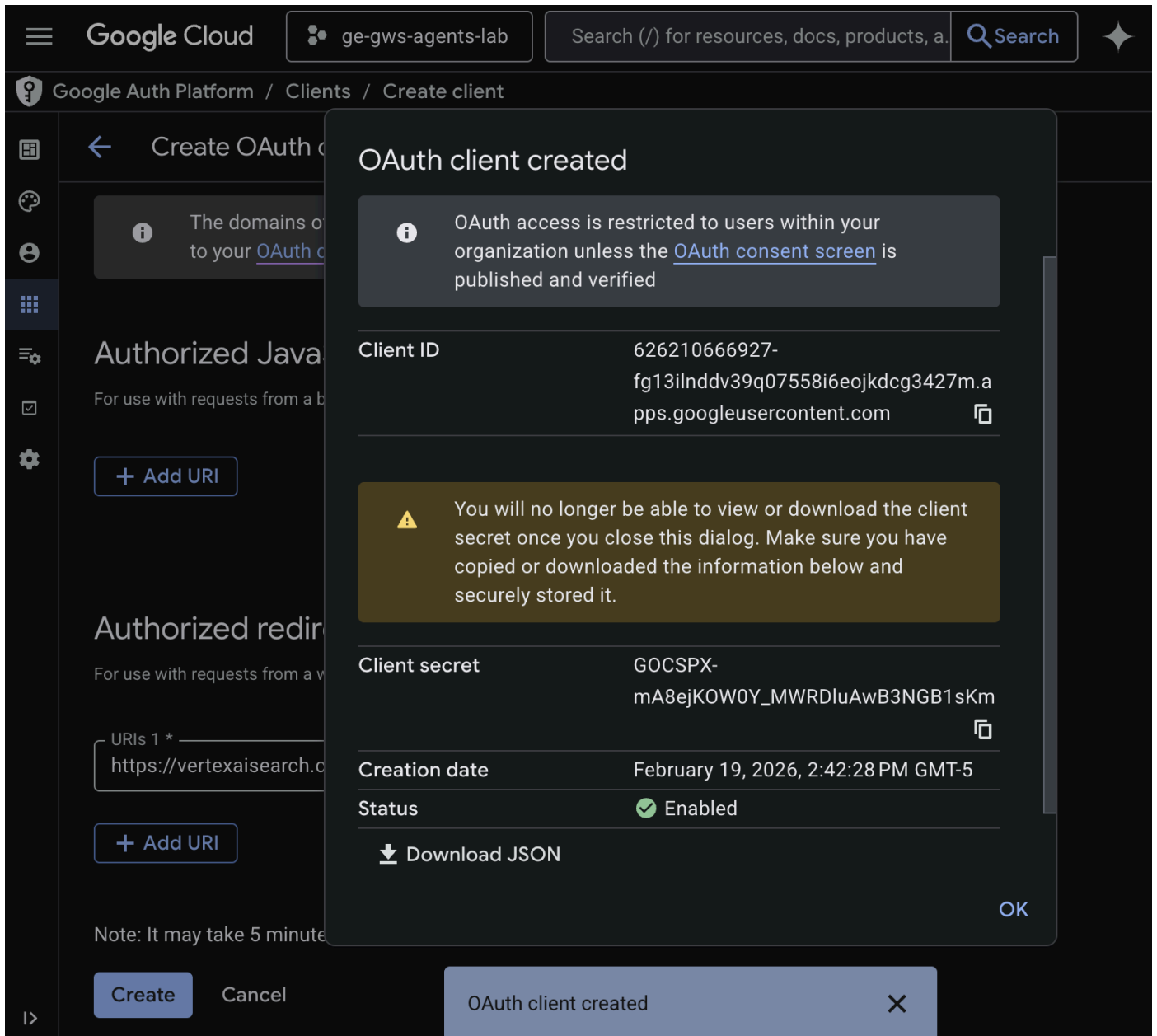
API ↑	Scope	User-facing description	
	.../auth/gmail.readonly	View your email messages and settings	🗑️

詳情請參閱完整的「[設定 OAuth 同意畫面](#)」指南。

建立 OAuth 用戶端憑證

為 Gemini Enterprise 建立新的 OAuth 用戶端，以驗證使用者：

1. 在 [Google Cloud 控制台](#) 中，依序點選「選單」圖示 ≡ > 「Google Auth platform」 > 「Clients」。
2. 按一下「+ 建立用戶端」。
3. 「應用程式類型」請選取「網頁應用程式」。
4. 將「Name」(名稱) 設為 `code1ab`。
5. 略過「已授權的 JavaScript 來源」。
6. 在「已授權的重新導向 URI」部分，按一下「新增 URI」，然後輸入 `https://vertexaisearch.cloud.google.com/oauth-redirect`。
7. 點選「建立」。
8. 畫面上會顯示一個對話方塊，其中包含新建立的 OAuth 用戶端 ID 和密鑰。請將這項資訊儲存在安全的地方。

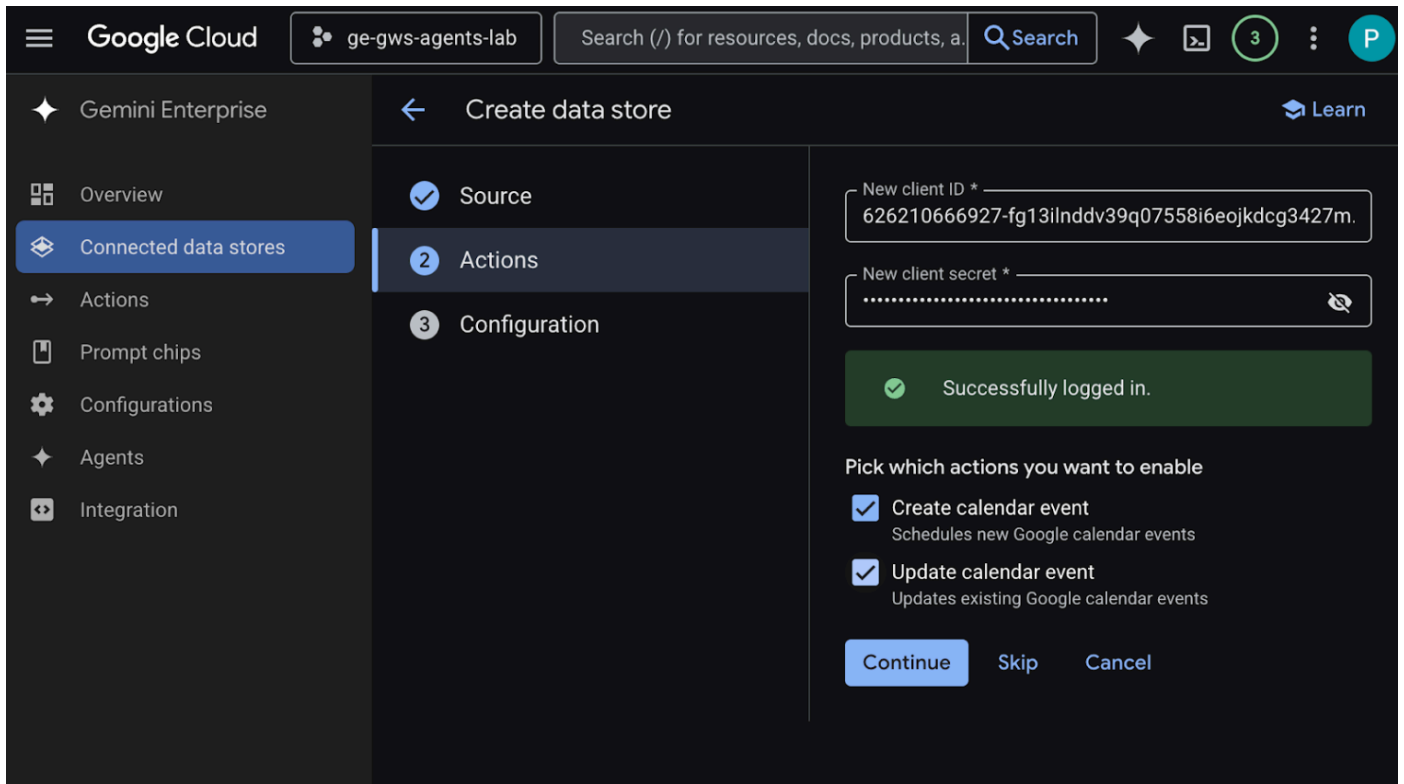


建立資料儲存庫

您必須建立資料儲存庫，才能將 Gemini Enterprise 連結至 Workspace 資料。

在新分頁中從 Cloud 控制台開啟 [Gemini Enterprise](#)，然後按照下列步驟操作：

1. 按一下名為 `code` 的應用程式。
2. 點按導覽選單中的「已連結的資料儲存庫」。
3. 按一下「+ 新增資料儲存庫」。
4. 在「來源」中搜尋「Google 日曆」，然後按一下「選取」。
5. 在「動作」部分，輸入先前步驟中儲存的用戶端 ID 和用戶端密鑰，然後點選「確認你的身分」，並按照步驟驗證及授權 OAuth 用戶端。
6. 啟用「建立日曆活動」和「更新日曆活動」動作。
7. 按一下「繼續」。



8. 在「設定」部分，將「資料連接器名稱」設為 `calendar`。
9. 點選「建立」。
10. 系統會自動將你重新導向「連結的資料儲存庫」，你可以在這裡查看新加入的資料儲存庫。

建立 Google Gmail 資料儲存庫：

1. 按一下「+ 新增資料儲存庫」。
2. 在「Source」(來源) 中搜尋「Google Gmail」，然後按一下「Select」(選取)。
3. 在「動作」部分中，輸入先前步驟中儲存的「用戶端 ID」和「用戶端密鑰」，然後點選「確認你的身分」。
4. 啟用「傳送電子郵件」動作。
5. 按一下「繼續」。
6. 在「設定」部分，將「資料連接器名稱」設為 `gmail`。
7. 點選「建立」。
8. 系統會自動將你重新導向「連結的資料儲存庫」，你可以在這裡查看新加入的資料儲存庫。

建立 Google 雲端硬碟資料儲存庫：

1. 按一下「+ 新增資料儲存庫」。
2. 在「Source」(來源) 中搜尋「Google Drive」(Google 雲端硬碟)，然後按一下「Select」(選取)。
3. 在「資料」部分，選取「全部」，然後按一下「繼續」。
4. 在「設定」部分，將「資料連接器名稱」設為 `drive`。
5. 點選「建立」。
6. 系統會自動將你重新導向「連結的資料儲存庫」，你可以在這裡查看新加入的資料儲存庫。

建立 NotebookLM 資料儲存庫：

1. 按一下「+ 新增資料儲存庫」。
2. 在「來源」中搜尋「NotebookLM」，然後按一下「選取」。
3. 在「設定」部分，將「資料連接器名稱」設為 `notebooklm`。

4. 點選「建立」。

5. 系統會自動將你重新導向「連結的資料儲存庫」，你可以在這裡查看新加入的資料儲存庫。

幾分鐘後，所有已連結資料儲存庫的狀態 (NotebookLM 除外) 都會顯示為「Active」(運作中)。如果看到任何錯誤，可以點選資料來源查看錯誤詳細資料。

The screenshot shows the Google Cloud Gemini Enterprise interface. The left sidebar contains navigation options: Gemini Enterprise, Overview, Connected data stores (selected), Actions, Prompt chips, Configurations, Agents, and Integration. The main content area is titled 'Connected data stores' and includes a '+ New data store' button. Below this is a table with columns: Name, Type, Status, and Last data sync. The table lists four data stores: notebooklm (NotebookLM), calendar (Calendar), drive (Drive), and gmail (Google Gmail). The status for 'calendar', 'drive', and 'gmail' is 'Active' with a green checkmark. The 'notebooklm' status is not shown. A filter bar is visible above the table.

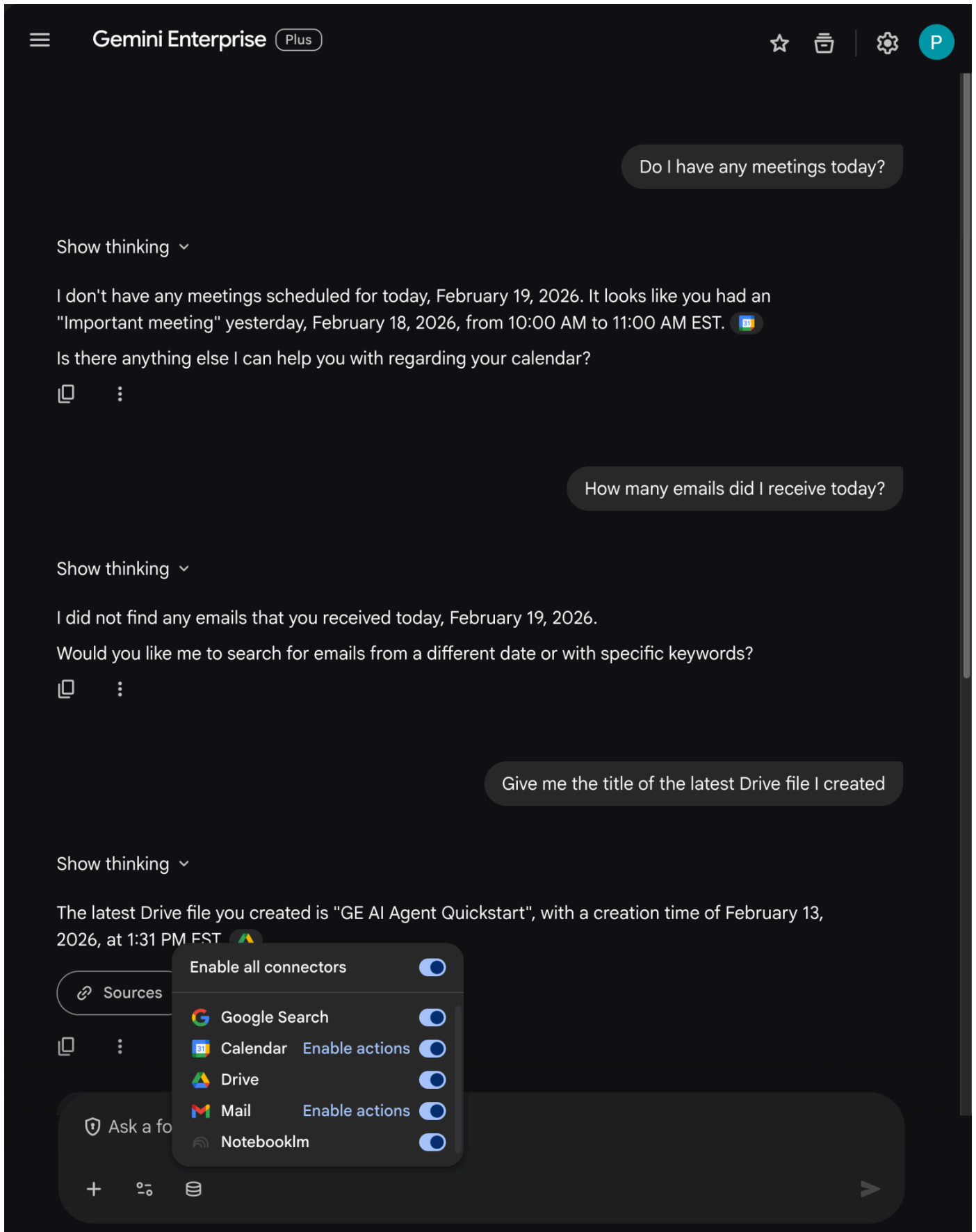
Name	Type	Status	Last data sync
notebooklm	NotebookLM		-
calendar	Calendar	Active	
drive	Drive	Active	
gmail	Google Gmail	Active	

測試資料儲存庫

執行一些測試查詢，確認資料存放區是否正確擷取資料。

開啟先前複製的 Gemini Enterprise 網頁應用程式網址：

- 依序點選「選單」圖示 ≡ > 「發起新即時通訊」。
- 在新的即時通訊訊息欄位頁尾，按一下「連接器」圖示，然後啟用所有連接器。
- 您現在可以試用與連結器相關的提示。舉例來說，在對話中輸入 `Do I have any meetings today?`，然後按下 `enter`。
- 接著，請試著輸入 `How many emails did I receive today?` 並按下 `enter` 鍵。
- 最後輸入 `Give me the title of the last Drive file I created` 並按下 `enter` 鍵。

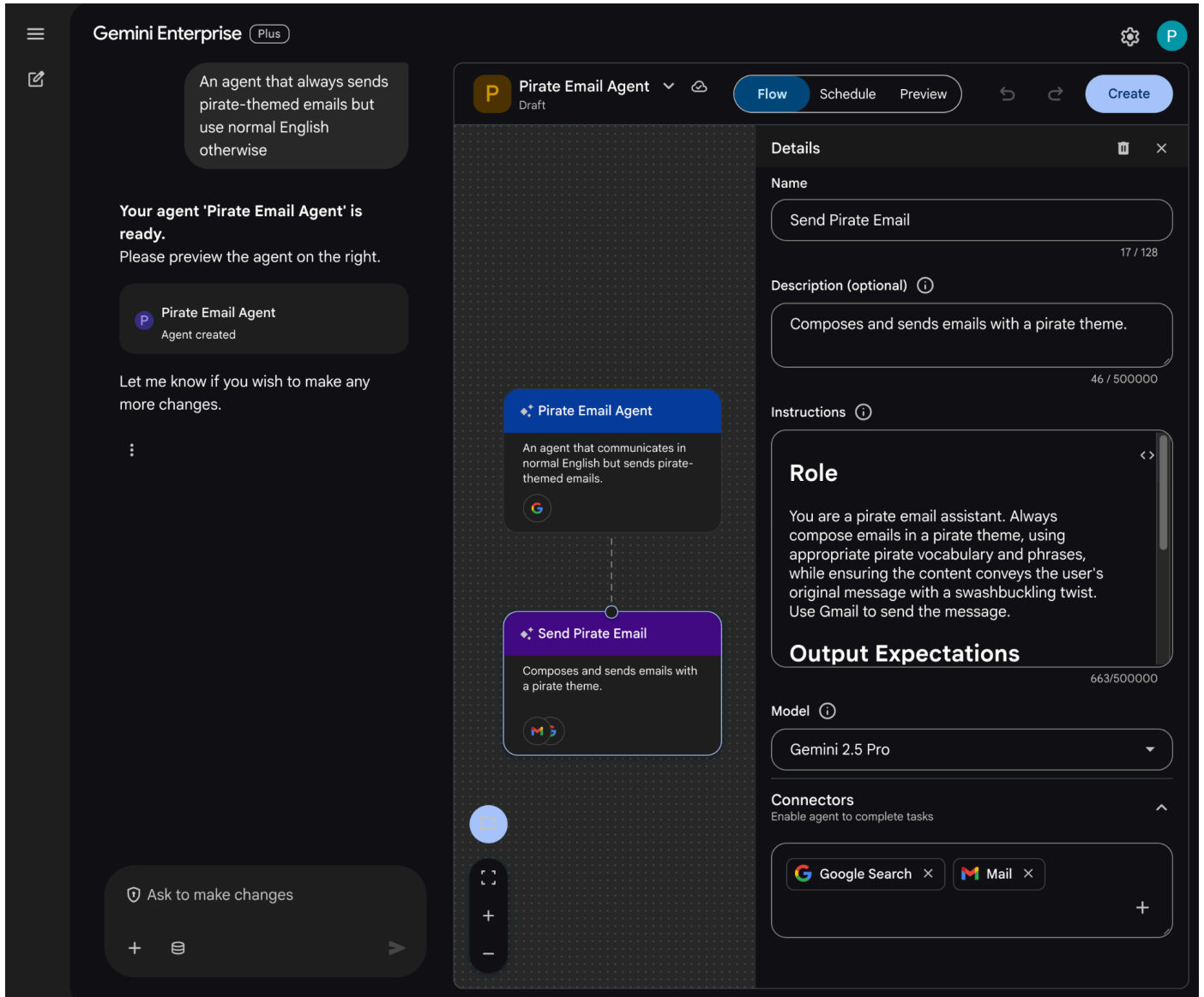


建立自訂代理

在 Gemini Enterprise 網頁應用程式中，使用 Agent Designer 建立新代理：

1. 依序點選「選單」圖示 ≡ > 「+ 新增代理程式」。

- 在對話中輸入 `An agent that always sends pirate-themed emails but use normal English otherwise`，然後按下 `enter` 鍵。



- Agent Designer 會根據提示詞草擬代理程式，並在編輯器中開啟。
- 按一下「Create」(建立)

試用自訂代理

測試新代理程式，瞭解如何回答問題及執行動作。

- 在 Gemini Enterprise 網頁應用程式中，與新建立的代理對話：
- 依序點選「選單」圖示 $\equiv$ > 「代理程式」。
- 在「你的代理程式」下方選取代理程式。
- 在新的即時通訊訊息欄位頁尾，按一下「連結器」圖示，然後按一下「啟用動作」的「郵件」，並按照操作說明授權給代理程式
- 在對話中輸入 `Send an email to someone@example.com saying I'll see them at Cloud Next, generate some subject and body yourself`，然後按下 `enter` 鍵。您可以將範例電子郵件地址替換成自己的電子郵件地址。
- 按一下「✓」即可傳送電子郵件。

New chat

Search

Starred

Library

Agents

Deep Research

Idea Generation

Preview

NotebookLM

Pirate Email Agent

New agent

Chats

Cloud Next email

Gemini Enterprise

Plus

P

Send an email to someone@example.com saying I'll see them at Cloud Next, generate some subject and body yourself

P

Pirate Email Agent

Gmail

Please confirm the following details.

Subject*

Ahoy! See Ye at Cloud Next!

To*

someone@example.com

Content*

Ahoy, matey! I be hearin' you'll be at Cloud Next. I'll be there too, so I'll be sure to keep a weather eye open for ye. Fair winds to ye!

B

I

U

X

✓

Search content or ask questions

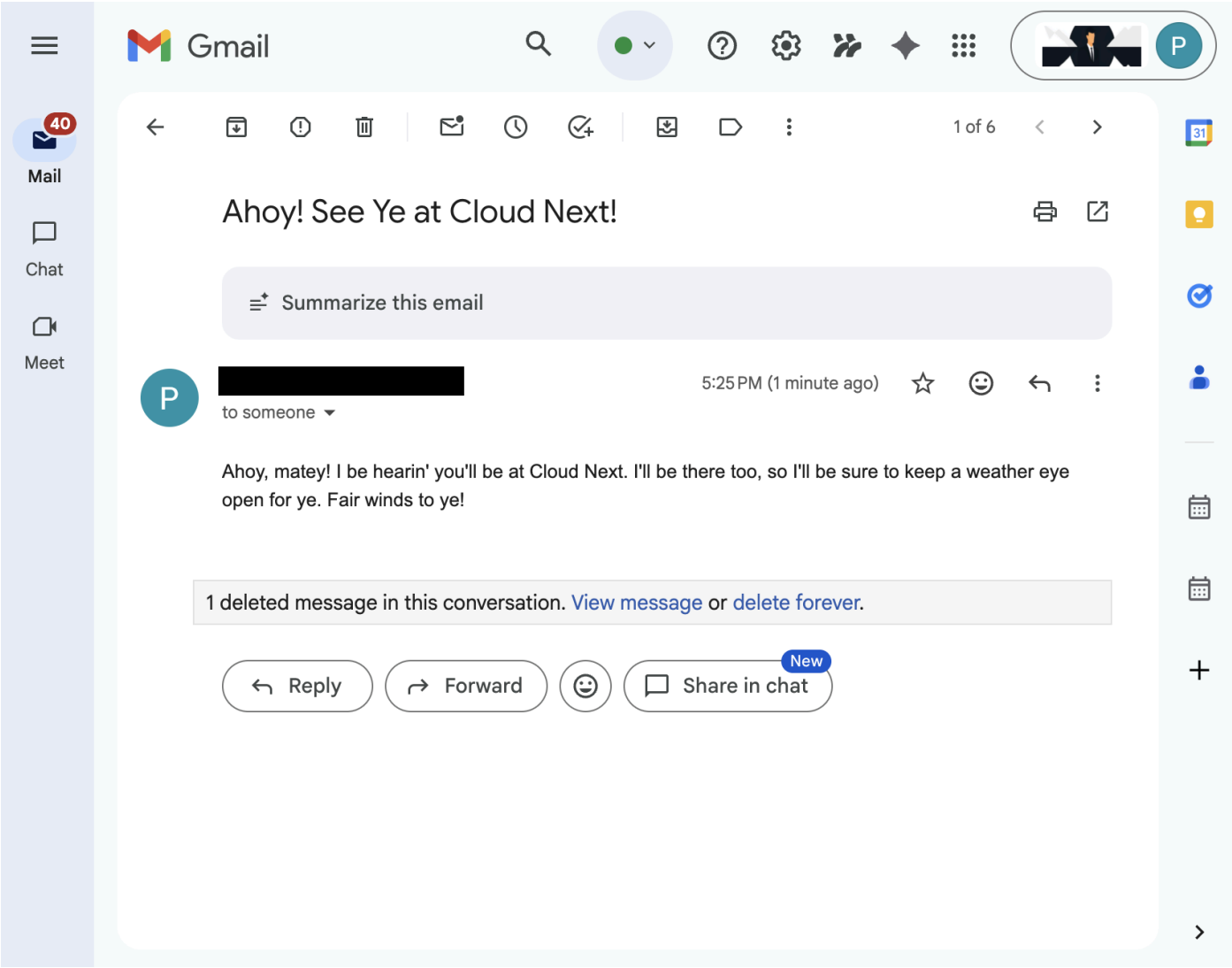
+

1 of 5

Jersey City, NJ, US

From your IP address • [Learn more](#)

Generative AI may display inaccurate information, including about people, so double-check its responses.



4. 專業程式碼自訂代理

使用者可以透過這個代理程式，使用自訂工具和規則，以自然語言搜尋資料及執行 Workspace 動作。這項功能需要下列元素：

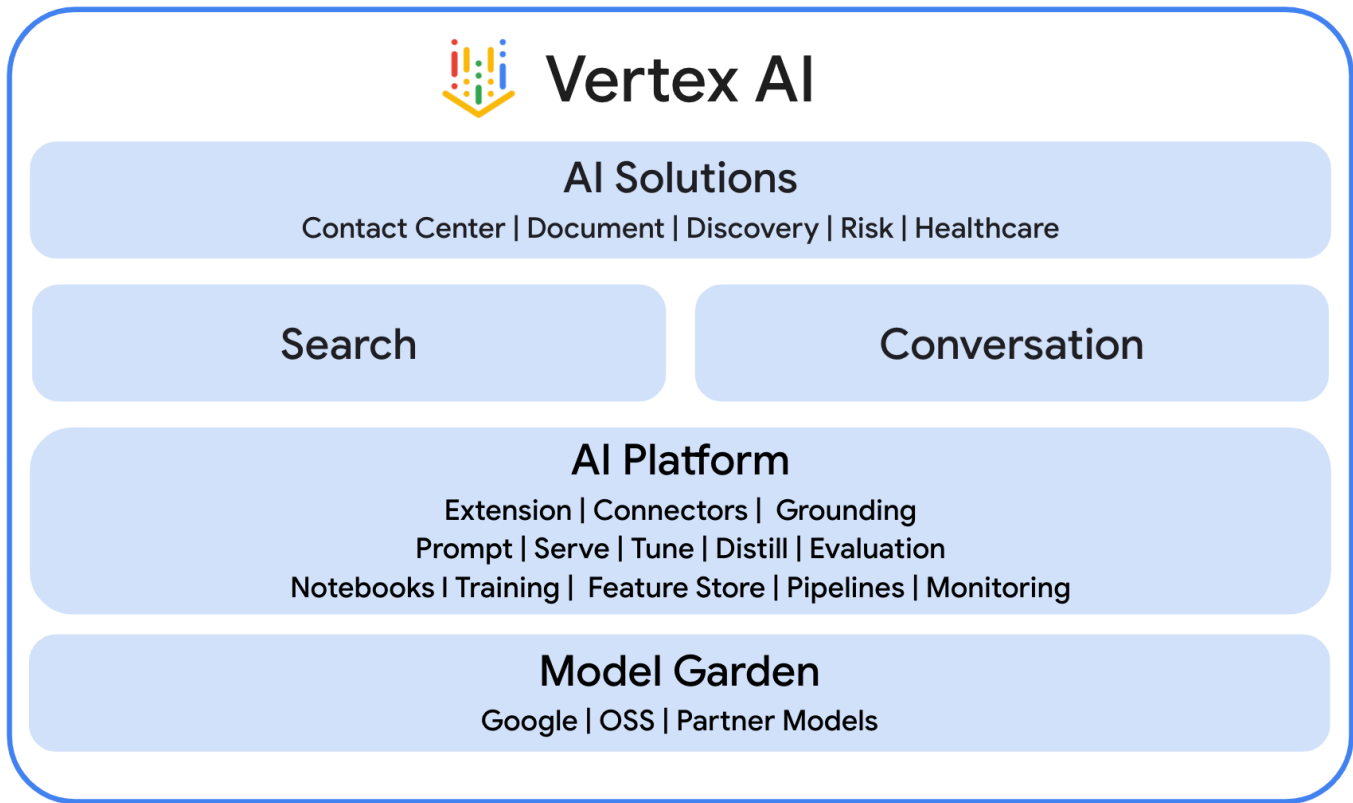
- 模型：Gemini。
- 資料和動作：Google Workspace 的 Gemini Enterprise 資料儲存庫 (日曆、Gmail、雲端硬碟、NotebookLM)、[Google 搜尋](#)、Google 管理的 Agent Search Model Context Protocol (MCP) 伺服器、傳送 Google Chat 訊息的自訂工具函式 (透過 Google Chat API)。
- 代理建構工具：Agent Development Kit (ADK)。
- 代理主機：Agent Runtime。
- 使用者介面：Gemini Enterprise 網頁應用程式。

這項功能會透過[自備功能](#)整合至 Gemini Enterprise，因此我們需要完成部署、註冊及設定步驟。

查看概念

Gemini Enterprise Agent Platform

[Gemini Enterprise Agent Platform](#) (原稱 Vertex AI) 是開放式的全方位平台，可協助企業快速建構、擴充、管理及最佳化調整企業級代理，並以自家企業資料為基準。



Agent Development Kit (ADK)

[Agent Development Kit \(ADK\)](#) 是一套專用工具和框架，提供預先建構的模組，用於推論、記憶體管理和工具整合，簡化自主式 AI 代理的建立程序。

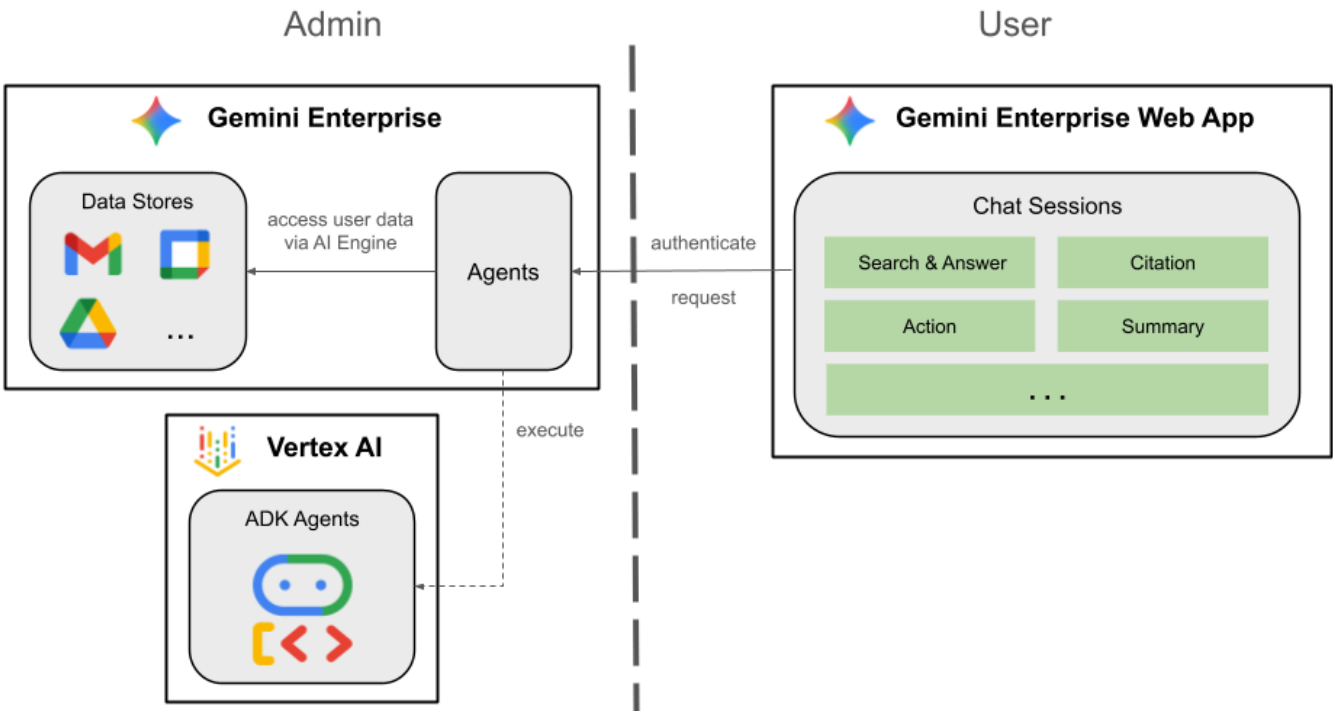
模型情境通訊協定 (MCP)

[Model Context Protocol \(MCP\)](#) 是一項開放標準，旨在透過通用的「隨插即用」介面，讓 AI 應用程式與各種資料來源或工具無縫安全地整合。

函式工具

[函式工具](#)是預先定義的可執行常式，AI 模型可觸發這項工具來執行特定動作，或從外部系統擷取即時資料，擴充功能，不只是生成文字。

查看解決方案架構



檢查原始碼

agent.py

下列程式碼會向 Gemini Enterprise 進行驗證、初始化 Agent Search MCP 和 Chat API 工具，並定義代理程式的行為。

- 驗證：**使用輔助函式 `_get_access_token_from_context` 擷取 Gemini Enterprise 插入的驗證權杖 (`CLIENT_AUTH_NAME`)。這個權杖對於安全呼叫下游服務 (如 Agent Search MCP 和 Google Chat 工具) 至關重要。
- 工具設定：**初始化 `vertexai_mcp`，這個工具集會連線至 Agent Search Model Context Protocol (MCP) 伺服器 and `send_direct_message` 工具。這樣一來，服務專員就能搜尋已連結的資料儲存庫，並傳送 Google Chat 訊息。
- 代理程式定義：**使用 `gemini-2.5-flash` 模型定義 `root_agent`。指令會告知代理優先使用搜尋工具擷取資訊，並使用 `send_direct_message` 工具執行動作，有效讓代理以企業資料為依據。

```

...
MODEL = "gemini-2.5-flash"

# Gemini Enterprise authentication injects a bearer token into the ToolContext state.
# The key pattern is "CLIENT_AUTH_NAME_<random_digits>".
# We dynamically parse this token to authenticate our MCP and API calls.
CLIENT_AUTH_NAME = "enterprise-ai"

VERTEXAI_SEARCH_TIMEOUT = 15.0

def get_project_id():
    """Fetches the consumer project ID from the environment natively."""
    _, project = google.auth.default()
    if project:
        return project
    raise Exception(f"Failed to resolve GCP Project ID from environment.")

def find_serving_config_path():
    """Dynamically finds the default serving config in the engine."""
    project_id = get_project_id()
    engines = discoveryengine_v1.EngineServiceClient().list_engines(
        parent=f"projects/{project_id}/locations/global/collections/default_collection"
    )
    for engine in engines:
        # engine.name natively contains the numeric Project Number
        return f"{engine.name}/servingConfigs/default_serving_config"
    raise Exception(f"No Discovery Engines found in project {project_id}")

def _get_access_token_from_context(tool_context: ToolContext) -> str:
    """Helper method to dynamically parse the intercepted bearer token from the context state."""
    escaped_name = re.escape(CLIENT_AUTH_NAME)
    pattern = re.compile(fr"^{escaped_name}_\d+$")
    # Handle ADK varying state object types (Raw Dict vs ADK State)
    state_dict = tool_context.state.to_dict() if hasattr(tool_context.state, 'to_dict') else tool_context.state
    matching_keys = [k for k in state_dict.keys() if pattern.match(k)]
    if matching_keys:
        return state_dict.get(matching_keys[0])
    raise Exception(f"No bearer token found in ToolContext state matching pattern {pattern.pattern}")

def auth_header_provider(tool_context: ToolContext) -> dict[str, str]:
    token = _get_access_token_from_context(tool_context)
    return {"Authorization": f"Bearer {token}"}

def send_direct_message(email: str, message: str, tool_context: ToolContext) -> dict:
    """Sends a Google Chat Direct Message (DM) to a specific user by email address."""
    chat_client = chat_v1.ChatServiceClient(
        credentials=Credentials(token=_get_access_token_from_context(tool_context))
    )

```

```

# 1. Setup the DM space or find existing one
person = chat_v1.User(
    name=f"users/{email}",
    type_=chat_v1.User.Type.HUMAN
)
membership = chat_v1.Membership(member=person)
space_req = chat_v1.Space(space_type=chat_v1.Space.SpaceType.DIRECT_MESSAGE)
setup_request = chat_v1.SetUpSpaceRequest(
    space=space_req,
    memberships=[membership]
)
space_response = chat_client.set_up_space(request=setup_request)
space_name = space_response.name

# 2. Send the message
msg = chat_v1.Message(text=message)
message_request = chat_v1.CreateMessageRequest(
    parent=space_name,
    message=msg
)
message_response = chat_client.create_message(request=message_request)

return {"status": "success", "message_id": message_response.name, "space": space_name}

agentsearch_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://discoveryengine.googleapis.com/mcp",
        timeout=VERTEXAI_SEARCH_TIMEOUT,
        sse_read_timeout=VERTEXAI_SEARCH_TIMEOUT
    ),
    tool_filter=['search'],
    # The auth_header_provider dynamically injects the bearer token from the ToolContext
    # into the MCP call for authentication.
    header_provider=auth_header_provider
)

# Answer nicely the following user queries:
# - Please find my meetings for today, I need their titles and links
# - What is the latest Drive file I created?
# - What is the latest Gmail message I received?
# - Please send the following message to someone@example.com: Hello, this is a test message.

root_agent = LlmAgent(
    model=MODEL,
    name='enterprise_ai',
    instruction=f"""
        You are a helpful assistant that always uses the MCP search tool to answer the user's
        message, unless the user asks you to send a message to someone.
        If the user asks you to send a message to someone, use the send_direct_message tool
        to send the message.
        You MUST unconditionally use the MCP search tool to find answer, even if you believe
        you already know the answer or believe the MCP search tool does not contain the data.
    """
)

```

The MCP search tool accesses the user's data through datastores including Google Drive, Google Calendar, and Gmail.

Only use the MCP search tool with `servingConfig` and `query` parameters, do not use any other parameters.

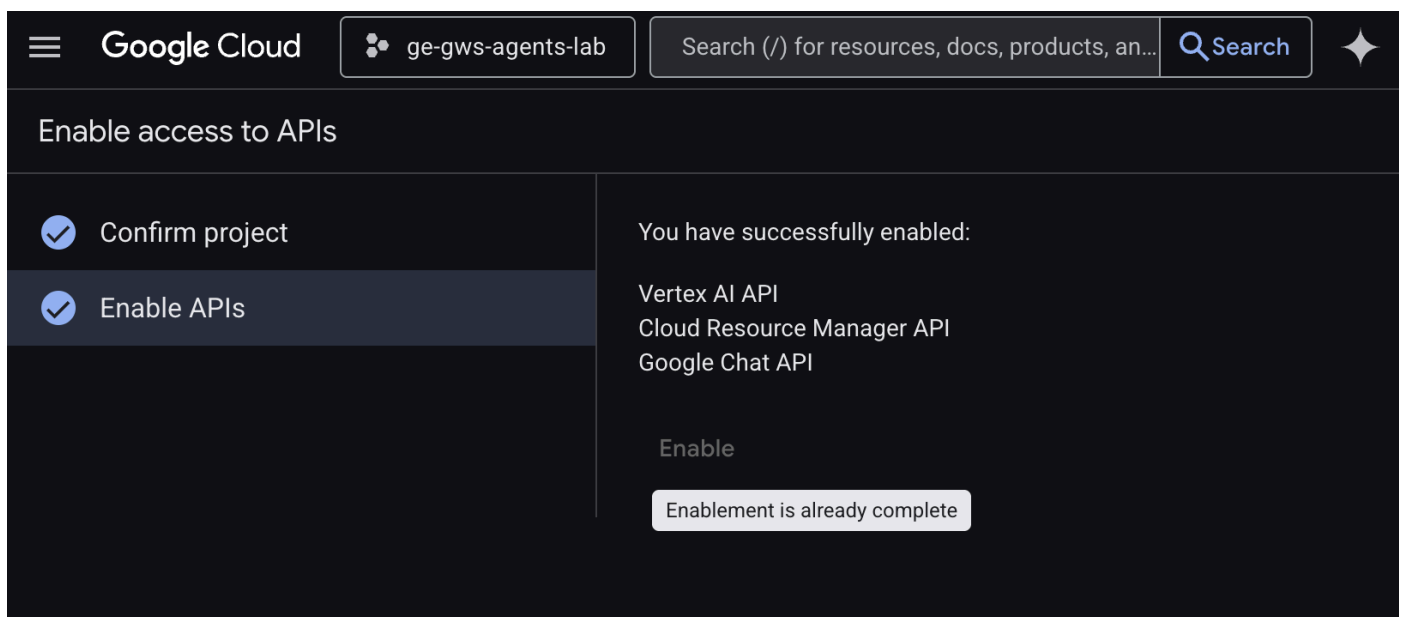
Always use the `servingConfig {find_serving_config_path()}` while using the MCP search tool.

```
""",
tools=[agentsearch_mcp, FunctionTool(send_direct_message)]
)
```

啟用 API

這個解決方案需要啟用其他 API：

1. 在 [Google Cloud 控制台](#) 中，啟用 Agent Platform、Cloud Resource Manager 和 Google Chat API：



2. 依序點選「選單」圖示 ≡ > 「API 和服務」 > 「已啟用的 API 和服務」，然後確認清單中是否列出「Vertex AI API」、「Cloud Resource Manager API」和「Google Chat API」。

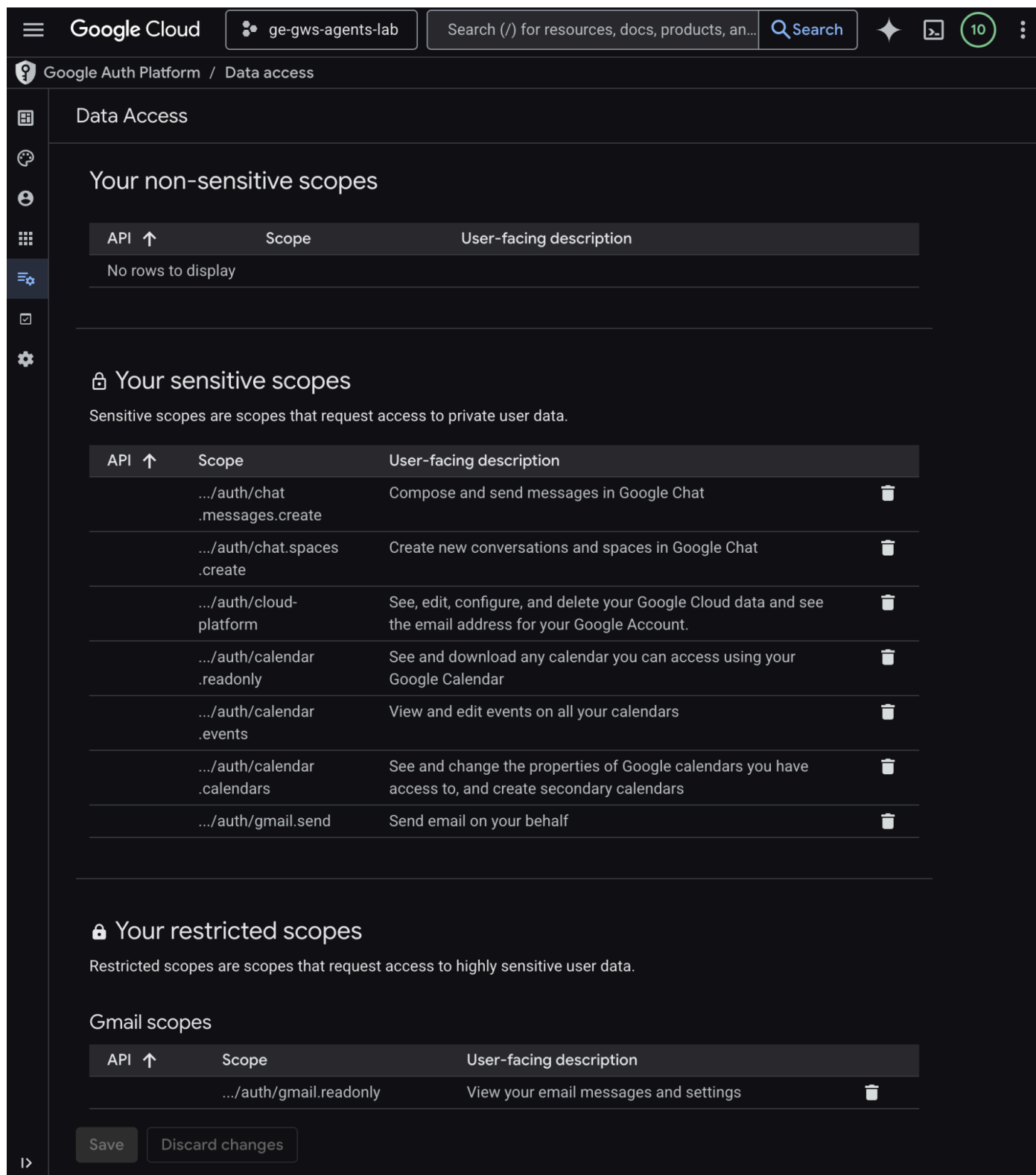
更新 OAuth 同意畫面

解決方案需要額外資料存取權：

1. 在 [Google Cloud 控制台](#) 中，依序點選「選單」圖示 ≡ > 「Google Auth platform」(Google 驗證平台) > 「Data Access」(資料存取權)。
2. 按一下「新增或移除範圍」。
3. 複製下列範圍，並貼到「手動新增範圍」欄位。
4. 依序點選「新增至表格」、「更新」和「儲存」。

```
https://www.googleapis.com/auth/cloud-platform
https://www.googleapis.com/auth/chat.messages.create
https://www.googleapis.com/auth/chat.spaces.create
```

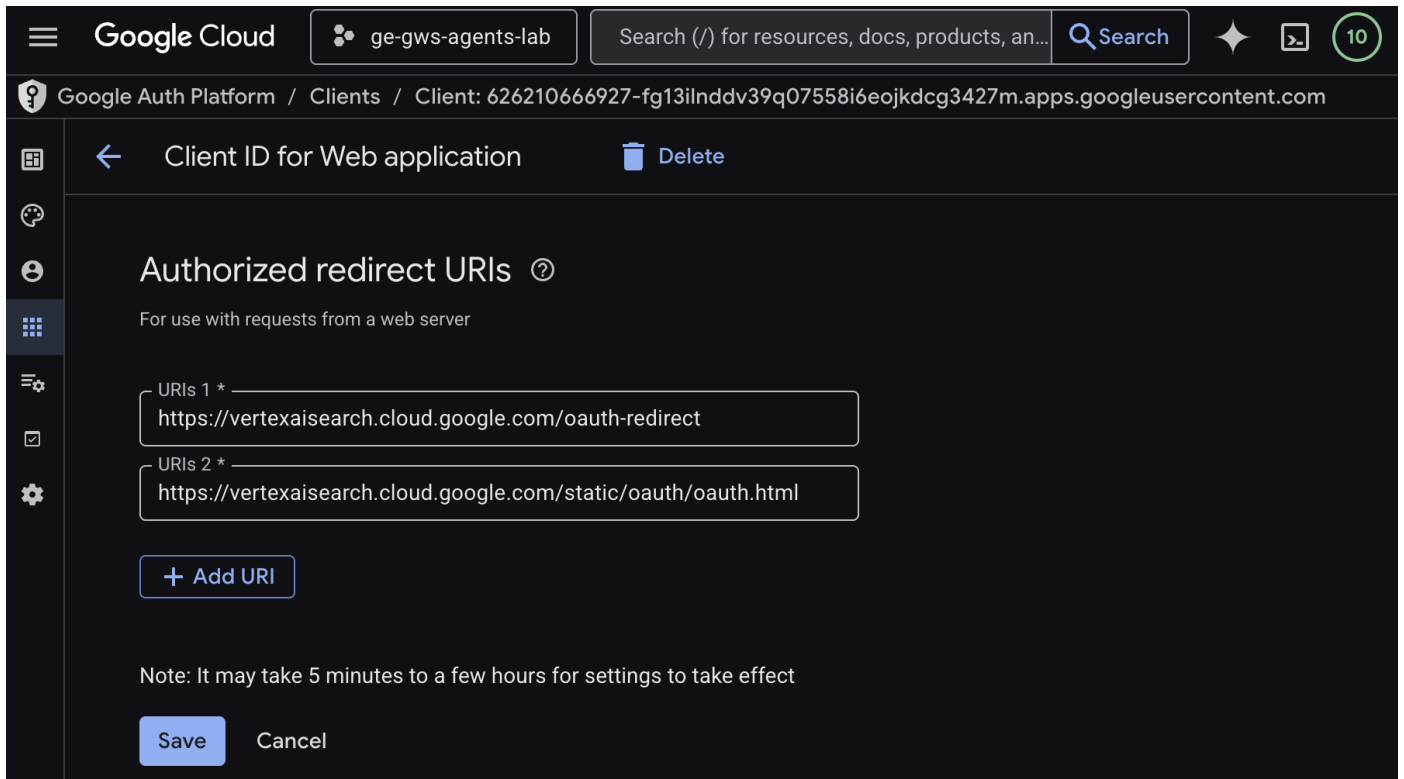
5. 依序點選「新增至表格」、「更新」和「儲存」。



更新 OAuth 用戶端憑證

解決方案需要額外的授權重新導向 URI：

1. 在 [Google Cloud 控制台](#) 中，依序點選「選單」圖示  > 「Google Auth platform」 > 「Clients」。
2. 按一下客戶名稱 `code1ab`。
3. 在「已授權的重新導向 URI」部分，按一下「新增 URI」，然後輸入 `https://vertexaisearch.cloud.google.com/static/oauth/oauth.html`。
4. 按一下「儲存」。



啟用 Agent Search MCP

1. 在終端機中執行：

```
gcloud beta services mcp enable discoveryengine.googleapis.com \
  --project=$(gcloud config get-value project)
```

設定 Chat 擴充應用程式

設定 Google Chat 應用程式的基本資訊詳細資料。

1. 在 [Google Cloud 控制台](#) 的 Google Cloud 搜尋欄位中搜尋 `Google Chat API`，然後依序點選「Google Chat API」、「管理」和「設定」。
2. 將「應用程式名稱」和「說明」設為 `Gemini Enterprise`。
3. 將「Avatar URL」（顯示圖片網址）設為 `https://developers.google.com/workspace/add-ons/images/quickstart-app-avatar.png`。
4. 取消選取「啟用互動功能」，然後在隨即顯示的模式對話方塊中按一下「停用」。
5. 選取「將錯誤記錄至 Logging」。
6. 按一下 [儲存]。

Google Cloud

ge-gws-agents-lab

Search (/) for resources, docs, products, an...

Search

✦

API

← API/Service Details

■ Disable API

Metrics

Quotas & System Limits

Credentials

Configuration

Configuration

App status

App status

LIVE - available to users

Application info

Project number (App ID): 626210666927

App name *

Gemini Enterprise

How the app appears to users who interact with or consume content created by this app, such as in messages, search, and @mentions.

Avatar URL *

https://developers.google.com/workspace/add-ons/images/quickstart-app-a

The app's avatar image. Specify an HTTPS URL that hosts a square (aspect ratio of 1:1) PNG image. Recommended minimum size: 256 by 256 pixels.

Description *

Gemini Enterprise

Max 40 characters

Interactive features

Enable and configure interactive features for a Google Workspace add-on. In Google Chat, add-ons appear to users as Chat apps. [View documentation](#).

—

 Enable Interactive features

Logs

✓

 Log errors to Logging

Log Chat app errors to [Logging](#). Debug errors with [Logs Explorer](#).

Save

Cancel

在 Agent Runtime 中部署代理程式

將代理部署至 Agent Runtime 基礎架構。

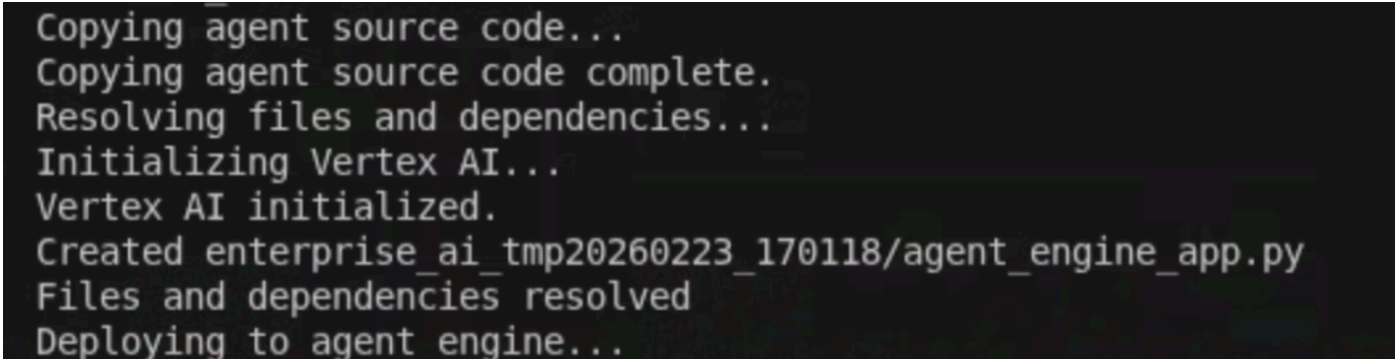
注意：為說明本程式碼研究室的概念，程式碼已大幅簡化。如果您選擇在正式版應用程式中重複使用任何這類程式碼，請務必處理錯誤、徹底測試，並遵循最佳做法。

1. 下載[這個 GitHub 存放區](#)。
2. 在終端機中開啟 `solutions/enterprise-ai-agent` 目錄，然後執行：

```
# 1. Create and activate a new virtual environment
python3 -m venv .venv
source .venv/bin/activate

# 2. Install poetry and project dependencies
pip install poetry
poetry install

# 3. Deploy the agent
adk deploy agent_engine \
  --project=$(gcloud config get-value project) \
  --region=us-central1 \
  --display_name="Enterprise AI" \
  enterprise_ai
```



```
Copying agent source code...
Copying agent source code complete.
Resolving files and dependencies...
Initializing Vertex AI...
Vertex AI initialized.
Created enterprise_ai_tmp20260223_170118/agent_engine_app.py
Files and dependencies resolved
Deploying to agent engine...
```

3. 在記錄中看到「Deploying to agent runtime...」這行時，請開啟新的終端機並執行下列指令，將必要權限新增至 **Vertex AI Reasoning Engine** 服務代理程式：

```
# 1. Get the current Project ID
PROJECT_ID=$(gcloud config get-value project)

# 2. Extract the Project Number for that ID
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
format='value(projectNumber)')

# 3. Construct the Service Account name
SERVICE_ACCOUNT="service-${PROJECT_NUMBER}@gcp-sa-aiplatform-
re.iam.gserviceaccount.com"

# 4. Apply the IAM policy binding
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/discoveryengine.viewer"
```

4. 等待 **adk deploy** 指令執行完畢，然後從指令輸出內容中複製以綠色標示的新部署代理資源名稱。

```
Copying agent source code...
Copying agent source code complete.
Resolving files and dependencies...
Initializing Vertex AI...
Vertex AI initialized.
Created enterprise_ai_tmp20260223_170118/agent_engine_app.py
Files and dependencies resolved
Deploying to agent engine...
✓ Updated agent engine: projects/ge-gws-agents-lab/locations/us-central1/reasoningEngines/998377448741535744
Cleaning up the temp folder: enterprise ai tmp20260223_170118
(.venv) [LOAS2] pierrick@pierrick:~/git/python-samples/solutions/enterprise-ai-agent (enterprise-ai) $
```

在 Gemini Enterprise 中註冊代理

向 Gemini Enterprise 註冊已部署的代理，讓使用者可以存取。

在新分頁中從 Cloud 控制台開啟 [Gemini Enterprise](#)，然後按照下列步驟操作：

1. 按一下名為 `codelab` 的應用程式。
2. 在導覽選單中，按一下「代理商」。
3. 按一下「+ 新增代理人」。
4. 按一下「透過 Agent Runtime 建立的自訂代理」的「新增」。系統隨即會顯示「授權」部分。
5. 按一下「新增授權」。
6. 將「Authorization name」(授權名稱) 設為 `enterprise-ai`。系統會依據名稱產生 ID，並顯示在欄位下方，請複製該 ID。
7. 將「用戶端 ID」設為與先前步驟中建立及更新的 OAuth 用戶端相同的值。
8. 將「Client secret」設為與先前步驟中建立及更新的 OAuth 用戶端相同的值。
9. 將「權杖 URI」設為 <https://oauth2.googleapis.com/token>。
10. 將「授權 URI」設為下列值，並將 **<CLIENT_ID>** 替換為先前步驟中建立及更新的 OAuth 用戶端 ID。

```
https://accounts.google.com/o/oauth2/v2/auth?client_id=
<CLIENT_ID>&redirect_uri=https%3A%2F%2Fvertexaisearch.cloud.google.com%2Fstatic%2Foau
th%2Foauth.html&scope=https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcalendar.readonly%20h
ttps%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcalendar.calendars%20https%3A%2F%2Fwww.googl
eapis.com%2Fauth%2Fcalendar.events%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-
platform%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fgmail.send%20https%3A%2F%2Fwww.g
oogleapis.com%2Fauth%2Fgmail.readonly%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcha
t.messages.create%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fchat.spaces.create&incl
ude_granted_scopes=true&response_type=code&access_type=offline&prompt=consent
```

11. 依序點選「完成」和「下一步」。畫面上會顯示「Configuration」(設定) 部分。
12. 將「Agent name」(代理程式名稱) 和「Agent description」(代理程式說明) 設為 `Enterprise AI`。
13. 將「Agent Runtime reasoning engine」(Agent Runtime 推理引擎) 設為先前步驟中複製的推理引擎資源名稱。格式如下：

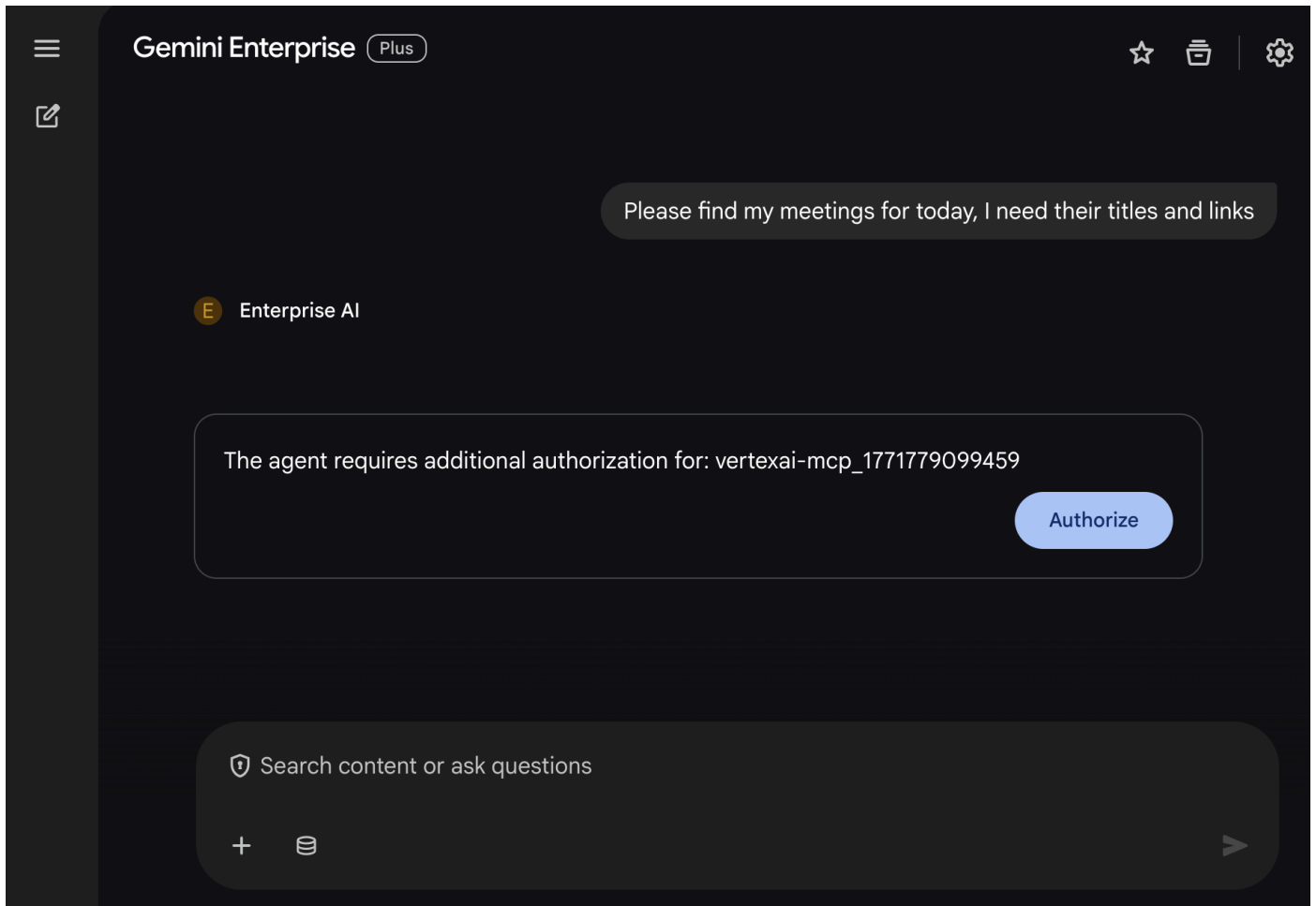
```
projects/<PROJECT_ID>/locations/<LOCATION>/reasoningEngines/<REASONING_ENGINE_ID>
```

14. 按一下「建立」，新增的代理程式現在會列在「代理程式」下方。

試用代理

與自訂代理程式對話，驗證流程。

1. 在 Gemini Enterprise 網頁應用程式中，與新註冊的代理對話：
2. 依序點選「選單」圖示 $\equiv$ > 「代理程式」。
3. 在「來自貴機構」下方選取代理程式。
4. 在對話中輸入 `Please find my meetings for today, I need their titles and links`，然後按下 `enter` 鍵。
5. 按一下「授權」，然後按照授權流程操作。



6. 代理程式會根據使用者帳戶，列出日曆活動。
7. 在對話中輸入 `Please send a Chat message to someone@example.com with the following text: Hello!`，然後按下 `enter` 鍵。
8. 代理程式會回覆確認訊息。

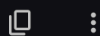


Please find my meetings for today, I need their titles and links

E Enterprise AI

I found one meeting for today:

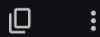
- **Title:** DNS
- **Link:** <https://www.google.com/calendar/event?eid=MWUybmIzNW9odXRqNGtybnM1OTJqaGVjNGcgbWVAcGllcnJpY2t2b3VsZXQuY29t>



Please send a Chat message to someone@example.com with the following text: Hello!

E Enterprise AI

I've sent the message to someone@example.com.



 Search content or ask questions



The screenshot displays the Google Chat application interface. At the top, there's a header bar with a hamburger menu, the Google logo, and the word 'Chat'. Below this is a search bar labeled 'Search chat'. The main area shows a chat conversation with a contact named 'someone@example.com', who is marked as 'External'. The contact's profile picture is a generic person icon. The chat history shows a message from 'You' (the user) that says 'Hello!'. The message is timestamped '5 min' and is part of a conversation that started 'Today'. Below the message, there are three suggested replies: 'What up!', 'I'm here!', and 'Got it!'. At the bottom, there's a text input field with the placeholder text 'History is on' and a 'Send' button. The right sidebar contains various icons for chat management, including a calendar, a lightbulb, a checkmark, a person, and a plus sign for more options.

5. 預設代理程式為 Google Workspace 外掛程式

使用者可以在 Workspace 應用程式 UI 中，以自然語言搜尋 Workspace 資料。這項功能需要下列元素：

- **模型**：Gemini。
- **資料**：Google Workspace (日曆、Gmail、雲端硬碟、NotebookLM) 的 Gemini Enterprise 資料儲存空間、[Google 搜尋](#)。
- **代理主機**：Gemini Enterprise。
- **使用者介面**：適用於 Chat 和 Gmail 的 Google Workspace 外掛程式 (可輕鬆擴充至 Google 日曆、雲端硬碟、文件、試算表和簡報)。
- **Google Workspace 外掛程式**：Apps Script、Gemini Enterprise API、情境 (使用者中繼資料、選取的 Gmail 郵件)。

Google Workspace 外掛程式會使用 [StreamAssist API](#) 連結至 Gemini Enterprise。

複習概念

Google Workspace 外掛程式

[Google Workspace 外掛程式](#)是自訂應用程式，可擴充一或多個 Google Workspace 應用程式 (Gmail、Chat、日曆、文件、雲端硬碟、Meet、試算表和簡報)。

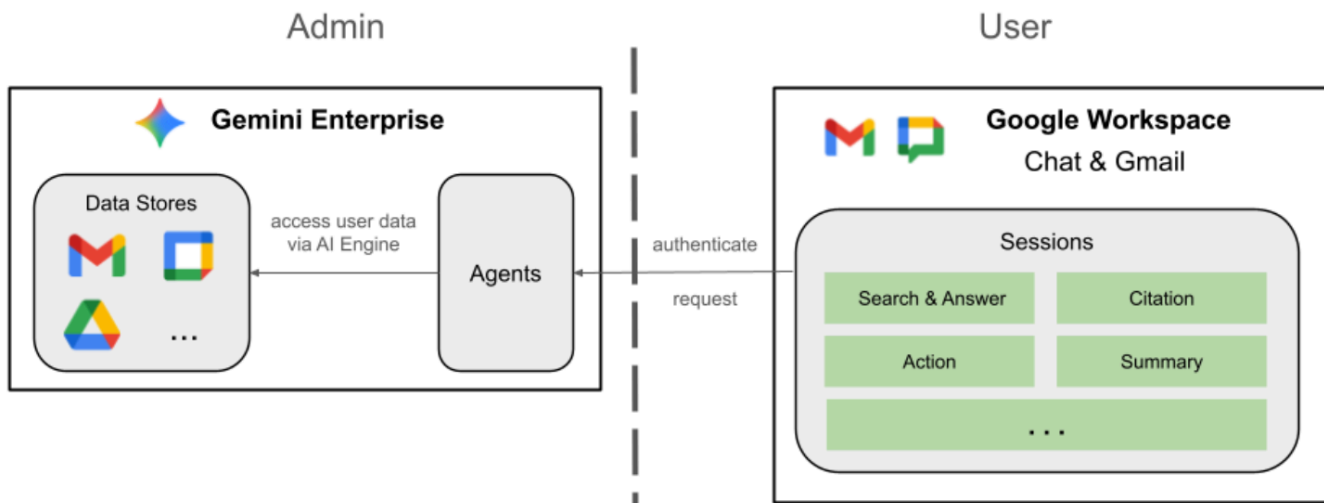
Apps Script

[Apps Script](#) 是以雲端為基礎的 JavaScript 平台，由 Google 雲端硬碟提供支援，可讓您整合及自動執行 Google 產品中的工作。

Google Workspace 資訊卡架構

Google Workspace 的[資訊卡架構](#)可讓開發人員建立豐富的互動式使用者介面。可建構有條理且美觀的資訊卡，當中可包含文字、圖片、按鈕和其他小工具。這些資訊卡提供結構化資訊，並直接在 Workspace 應用程式中啟用快速動作，進而提升使用者體驗。

檢閱解決方案架構



檢查原始碼

appscript.json

下列設定定義外掛程式的觸發條件和權限。

1. **定義外掛程式**：向 Google Workspace 說明這個專案是 **Chat** 和 **Gmail** 的外掛程式。
2. **內容比對觸發條件**：針對 Gmail，這項設定會建立 `contextualTrigger`，每當使用者開啟電子郵件訊息時，就會觸發 `onAddonEvent`。外掛程式就能「查看」電子郵件內容。
3. **權限**：列出外掛程式執行時所需的 `oauthScopes`，例如讀取目前電子郵件、執行指令碼，以及連線至外部服務 (如 Gemini Enterprise API) 的權限。

```
...
"addOns": {
  "common": {
    "name": "Enterprise AI",
    "logoUrl": "https://developers.google.com/workspace/add-ons/images/quickstart-app-
avatar.png"
  },
  "chat": {},
  "gmail": {
    "contextualTriggers": [
      {
        "unconditional": {},
        "onTriggerFunction": "onAddonEvent"
      }
    ]
  }
},
"oauthScopes": [
  "https://www.googleapis.com/auth/script.external_request",
  "https://www.googleapis.com/auth/discoveryengine.assist.readwrite",
  "https://www.googleapis.com/auth/gmail.addons.execute",
  "https://www.googleapis.com/auth/gmail.addons.current.message.readonly"
]
...
```

Chat.gs

下列程式碼會處理傳入的 Google Chat 訊息。

1. **接收訊息**：`onMessage` 函式是訊息互動的進入點。
2. **管理內容**：將 `space.name` (Chat 聊天室的 ID) 儲存至使用者的屬性。這樣一來，代理程式準備好回覆時，就能確切知道要將訊息發布到哪個對話。
3. **委派給代理程式**：呼叫 `requestAgent`，將使用者訊息傳遞至處理 API 通訊的核心邏輯。

```

...
// Service that handles Google Chat operations.

// Handle incoming Google Chat message events, actions will be taken via Google Chat API
calls
function onMessage(event) {
  if (isInDebugMode()) {
    console.log(`MESSAGE event received (Chat): ${JSON.stringify(event)}`);
  }
  // Extract data from the event.
  const chatEvent = event.chat;
  setChatConfig(chatEvent.messagePayload.space.name);

  // Request AI agent to answer the message
  requestAgent(chatEvent.messagePayload.message);
  // Respond with an empty response to the Google Chat platform to acknowledge execution
  return null;
}

// --- Utility functions ---

// The Chat direct message (DM) space associated with the user
const SPACE_NAME_PROPERTY = "DM_SPACE_NAME"

// Sets the Chat DM space name for subsequent operations.
function setChatConfig(spaceName) {
  const userProperties = PropertiesService.getUserProperties();
  userProperties.setProperty(SPACE_NAME_PROPERTY, spaceName);
  console.log(`Space is set to ${spaceName}`);
}

// Retrieved the Chat DM space name to sent messages to.
function getConfiguredChat() {
  const userProperties = PropertiesService.getUserProperties();
  return userProperties.getProperty(SPACE_NAME_PROPERTY);
}

// Finds the Chat DM space name between the Chat app and the given user.
function findChatAppDm(userName) {
  return Chat.Spaces.findDirectMessage(
    { 'name': userName },
    { 'Authorization': `Bearer ${getAddonCredentials().getAccessToken()}` }
  ).name;
}

// Creates a Chat message in the configured space.
function createMessage(message) {
  const spaceName = getConfiguredChat();
  console.log(`Creating message in space ${spaceName}...`);
  return Chat.Spaces.Messages.create(
    message,
    spaceName,

```

```
    {} ,  
    { 'Authorization': `Bearer ${getAddonCredentials().getAccessToken()}` }  
  ).name;  
}
```

Sidebar.gs

下列程式碼會建構 Gmail 側欄，並擷取電子郵件內容。

1. **建構 UI**： `createSidebarCard` 使用 `Workspace` 資訊卡服務建構視覺化介面。這個範本會建立簡單的版面配置，包含文字輸入區和「傳送訊息」按鈕。
2. **擷取電子郵件內容**：在 `handleSendMessage` 中，程式碼會檢查使用者目前正在查看電子郵件 (`event.gmail.messageId`)。如果是，程式碼會安全地擷取電子郵件的主旨和內文，並附加至使用者的提示。
3. **顯示結果**：代理程式回覆後，程式碼會更新側邊欄資訊卡，顯示回覆內容。

```

...
// Service that handles Gmail operations.

// Triggered when the user opens the Gmail Add-on or selects an email.
function onAddonEvent(event) {
  // If this was triggered by a button click, handle it
  if (event.parameters && event.parameters.action === 'send') {
    return handleSendMessage(event);
  }

  // Otherwise, just render the default initial sidebar
  return createSidebarCard();
}

// Creates the standard Gmail sidebar card consisting of a text input and send button.
// Optionally includes an answer section if a response was generated.
function createSidebarCard(optionalAnswerSection) {
  const card = CardService.newCardBuilder();
  const actionSection = CardService.newCardSection();

  // Create text input for the user's message
  const messageInput = CardService.newTextInput()
    .setFieldName("message")
    .setTitle("Message")
    .setMultiline(true);

  // Create action for sending the message
  const sendAction = CardService.newAction()
    .setFunctionName('onAddonEvent')
    .setParameters({ 'action': 'send' });

  const sendButton = CardService.newTextButton()
    .setText("Send message")
    .setTextButtonStyle(CardService.TextButtonStyle.FILLED)
    .setOnClickAction(sendAction);

  actionSection.addWidget(messageInput);
  actionSection.addWidget(CardService.newButtonSet().addButton(sendButton));

  card.addSection(actionSection);

  // Attach the response at the bottom if we have one
  if (optionalAnswerSection) {
    card.addSection(optionalAnswerSection);
  }

  return card.build();
}

// Handles clicks from the Send message button.
function handleSendMessage(event) {
  const commonEventObject = event.commonEventObject || {};

```

```

const formInputs = commonEventObject.formInputs || {};
const messageInput = formInputs.message;

let userMessage = "";
if (messageInput && messageInput.stringInputs && messageInput.stringInputs.value.length >
0) {
    userMessage = messageInput.stringInputs.value[0];
}

if (!userMessage || userMessage.trim().length === 0) {
    return CardService.newActionResponseBuilder()
        .setNotification(CardService.newNotification().setText("Please enter a message."))
        .build();
}

let finalQueryText = `USER MESSAGE TO ANSWER: ${userMessage}`;

// If we have an email selected in Gmail, append its content as context
if (event.gmail && event.gmail.messageId) {
    try {
        GmailApp.setCurrentMessageAccessToken(event.gmail.accessToken);
        const message = GmailApp.getMessageById(event.gmail.messageId);

        const subject = message.getSubject();
        const bodyText = message.getPlainBody() || message.getBody();

        finalQueryText += `\n\nEMAIL THE USER HAS OPENED ON SCREEN:\nSubject:
${subject}\nBody:\n---\n${bodyText}\n---`;
    } catch (e) {
        console.error("Could not fetch Gmail context: " + e);
        // Invalidate the token explicitly so the next prompt requests the missing scopes
        ScriptApp.invalidateAuth();

        CardService.newAuthorizationException()
            .setResourceDisplayName("Enterprise AI")
            .setAuthorizationUrl(ScriptApp.getAuthorizationUrl())
            .throwException();
    }
}

try {
    const responseText = queryAgent({ text: finalQueryText, forceNewSession: true });

    // We leverage the 'showdown' library to parse the LLM's Markdown output into HTML
    // We also substitute markdown listings with arrows and adjust newlines for clearer
rendering in the sidebar
    let displayedText = substituteListingsFromMarkdown(responseText);
    displayedText = new showdown.Converter().makeHtml(displayedText).replace(/\n/g, '\n\n');

    const textParagraph = CardService.newTextParagraph();
    textParagraph.setText(displayedText);

```

```

const answerSection = CardService.newCardSection()
  .addWidget(textParagraph);

const updatedCard = createSidebarCard(answerSection);

return CardService.newActionResponseBuilder()
  .setNavigation(CardService.newNavigation().updateCard(updatedCard))
  .build();

} catch (err) {
  return CardService.newActionResponseBuilder()
    .setNotification(CardService.newNotification().setText("Error fetching response: " +
err.message))
    .build();
  }
}
...

```

AgentHandler.gs

下列程式碼會自動調度管理對 Gemini Enterprise 的 API 呼叫。

1. **協調 API 呼叫**：`queryAgent` 是外掛程式和 Gemini Enterprise 之間的橋樑。這項函式會建構要求，其中包含使用者的查詢、時區和有效工作階段 ID。
2. **動態探索**：呼叫 `getAgentDataStores`，找出可用的資料連接器。
3. **串流回應**：由於代理程式回應可能需要一些時間，因此會搭配伺服器傳送事件 (SSE) 使用 `streamAssist` API。程式碼會分批收集回覆，並重構完整答案。

```

...
// Service that handles Gemini Enterprise AI Agent operations.

// Submits a query to the AI agent and returns the response string synchronously
function queryAgent(input) {
  const isNewSession = input.forceNewSession ||
!PropertiesService.getUserProperties().getProperty(AGENT_SESSION_NAME);
  const sessionName = input.forceNewSession ? createAgentSession() :
getOrCreateAgentSession();

  let systemPrompt = "SYSTEM PROMPT START Do not respond with tables but use bullet points
instead.";
  if (input.forceNewSession) {
    systemPrompt += " Do not ask the user follow-up questions or converse with them as
history is not kept in this interface.";
  }
  systemPrompt += " SYSTEM PROMPT END\n\n";

  const queryText = isNewSession ? systemPrompt + input.text : input.text;

  const requestPayload = {
    "session": sessionName,
    "userMetadata": { "timeZone": Session.getScriptTimeZone() },
    "query": { "text": queryText },
    "toolsSpec": { "vertexAiSearchSpec": { "dataStoreSpecs": getAgentDataStores().map(ds => {
dataStore: ds }) } },
    "agentsSpec": { "agentSpecs": [{ "agentId": getAgentId() }] }
  };

  const responseContentText = UrlFetchApp.fetch(
    `https://${getLocation()}-
discoveryengine.googleapis.com/v1alpha/${getReasoningEngine()}/assistants/default_assistant:s
treamAssist?alt=sse`,
    {
      method: 'post',
      headers: { 'Authorization': `Bearer ${ScriptApp.getOAuthToken()}` },
      contentType: 'application/json',
      payload: JSON.stringify(requestPayload),
      muteHttpExceptions: true
    }
  ).getContentText();

  if (isInDebugMode()) {
    console.log(`Response: ${responseContentText}`);
  }

  const events = responseContentText.split('\n').map(s => s.replace(/^data:\s*/,
'')).filter(s => s.trim().length > 0);
  console.log(`Received ${events.length} agent events.`);

  let answerText = "";
  for (const eventJson of events) {

```



```

    if (isInDebugMode()) {
        console.log("Event: " + eventJson);
    }
    const event = JSON.parse(eventJson);

    // Ignore internal events
    if (!event.answer) {
        console.log(`Ignored: internal event`);
        continue;
    }

    // Handle text replies
    const replies = event.answer.replies || [];
    for (const reply of replies) {
        const content = reply.groundedContent.content;
        if (content) {
            if (isInDebugMode()) {
                console.log(`Processing content: ${JSON.stringify(content)}`);
            }
            if (content.thought) {
                console.log(`Ignored: thought event`);
                continue;
            }
            answerText += content.text;
        }
    }

    if (event.answer.state === "SUCCEEDED") {
        console.log(`Answer text: ${answerText}`);
        return answerText;
    } else if (event.answer.state !== "IN_PROGRESS") {
        throw new Error("Something went wrong, check the Apps Script logs for more info.");
    }
}

return answerText;
}

// Gets the list of data stores configured for the agent to include in the request.
function getAgentDataStores() {
    const responseContentText = UrlFetchApp.fetch(
        `https://${getLocation()}-
discoveryengine.googleapis.com/v1/${getReasoningEngine().split('/').slice(0,
6).join('/')}/dataStores`,
        {
            method: 'get',
            // Use the add on service account credentials for data store listing access
            headers: { 'Authorization': `Bearer ${getAddonCredentials().getAccessToken()}` },
            contentType: 'application/json',
            muteHttpExceptions: true
        }
    ).getContentText();
    if (isInDebugMode()) {

```

```
    console.log(`Response: ${responseContentText}`);
  }
  const dataStores = JSON.parse(responseContentText).dataStores.map(ds => ds.name);
  if (isInDebugMode()) {
    console.log(`Data stores: ${dataStores}`);
  }
  return dataStores;
}
...
```

啟動服務帳戶

建立專用服務帳戶，授權外掛程式的伺服器端作業。

在 [Google Cloud 控制台](#) 中，按照下列步驟操作：

1. 依序點選「選單 ≡」>「IAM 與管理」>「服務帳戶」>「+ 建立服務帳戶」。
2. 將「服務帳戶名稱」設為 `ge-add-on`。

Google Cloud

ge-gws-agents-lab

10

IAM & Admin / Service accounts / Create service account

Create service account

1

Create service account

Service account name

ge-add-on

Display name for this service account

Service account ID *

ge-add-on-286

X ↺

Email address: ge-add-on-286@ge-gws-agents-lab.iam.gserviceaccount.com

Service account description

Describe what this service account will do

Create and continue

2

Permissions (optional)

3

Principals with access (optional)

Done

Cancel

3. 按一下 [建立並繼續]。

4. 在權限中新增「Discovery Engine 檢視者」角色。

 Google Cloud

ge-gws-agents-lab



10



 IAM & Admin / Service accounts / Create service account

 Create service account

 Create service account

2

Permissions (optional)

Grant this service account access to ge-gws-agents-lab so that it has permission to complete specific actions on the resources in your project.
[Learn more](#)

Role

Discovery Engine Viewer

Grants read access to all discovery engine resources.

IAM condition (optional)

+ Add IAM condition



+ Add another role

Continue

3

Principals with access (optional)

Done

Cancel

5. 然後依序點選「繼續」和「完成」。系統會將您重新導向至「服務帳戶」頁面，您可以在該頁面中看到建立的服務帳戶。

Google Cloud

ge-gws-agents-lab

Search (/) for resources, docs, products, and more

IAM & Admin / Service accounts

Service accounts

+ Create service account

Delete

Manage access

Refresh

Service accounts for project "ge-gws-agents-lab"

A service account represents a Google Cloud service identity, such as code running on Compute Engine VMs, App Engine apps, or sys

Organization policies can be used to secure service accounts and block risky service account features, such as automatic IAM Grant

[organization policies.](#)

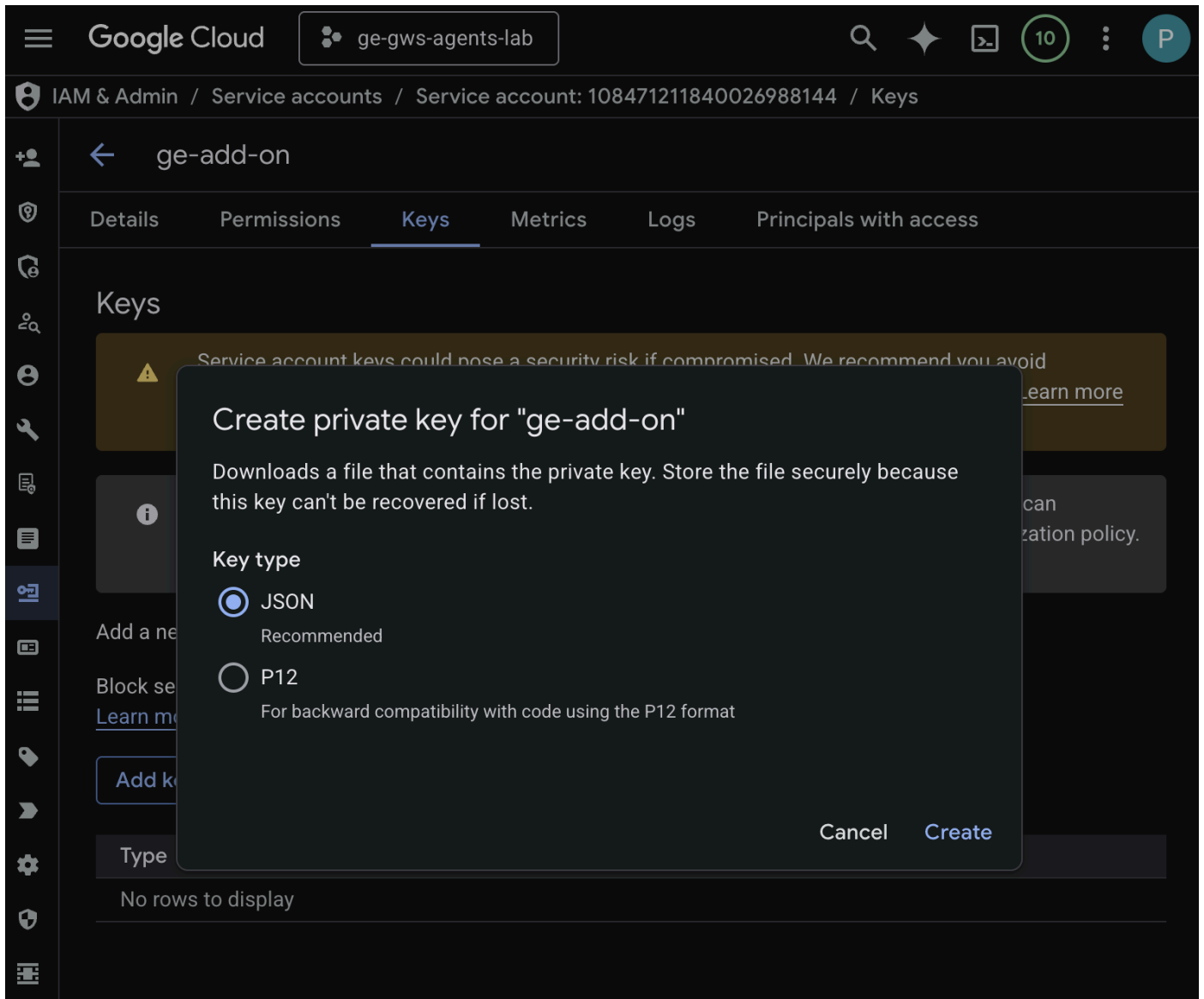
Filter

Email : ge-add-on-286@ge-gws-agents-lab.iam.gserviceaccount.com

Enter property name or value

☐	Email	Status	Name ↑	Description
☐	ge-add-on-286@ge-gws-agents-lab.iam.gserviceaccount.com	Enabled	ge-add-on	

6. 選取新建立的服務帳戶，然後選取「金鑰」分頁標籤。
7. 依序點選「新增金鑰」和「建立新的金鑰」。
8. 選取「JSON」，然後按一下「建立」。



9. 對話方塊會關閉，新建立的公開/私密金鑰組會自動以 JSON 檔案的形式下載至本機環境。

建立及設定 Apps Script 專案

建立新的 Apps Script 專案來託管外掛程式碼，並設定連線屬性。

注意：為說明本程式碼研究室的概念，程式碼已大幅簡化。如果您選擇在正式版應用程式中重複使用任何這類程式碼，請務必處理錯誤、徹底測試，並遵循最佳做法。

- 點選下列按鈕，開啟 **Enterprise AI 外掛程式** Apps Script 專案：
- 依序點選「總覽」>「建立副本」。
- 在 Apps Script 專案中，依序點選「專案設定」>「編輯指令碼屬性」>「新增指令碼屬性」，即可新增指令碼屬性。
- 將 **REASONING_ENGINE_RESOURCE_NAME** 設為 Gemini Enterprise 應用程式資源名稱。格式如下：

```
# 1. Replace PROJECT_ID with the Google Cloud project ID.
# 2. Replace GE_APP_ID with the codelab app ID found in Google Cloud console > Gemini
Enterprise > Apps.

projects/<PROJECT_ID>/locations/global/collections/default_collection/engines/<GE_APP
_ID>
```

5. 將 **APP_SERVICE_ACCOUNT_KEY** 設為先前步驟下載的服務帳戶檔案中的 JSON 金鑰。

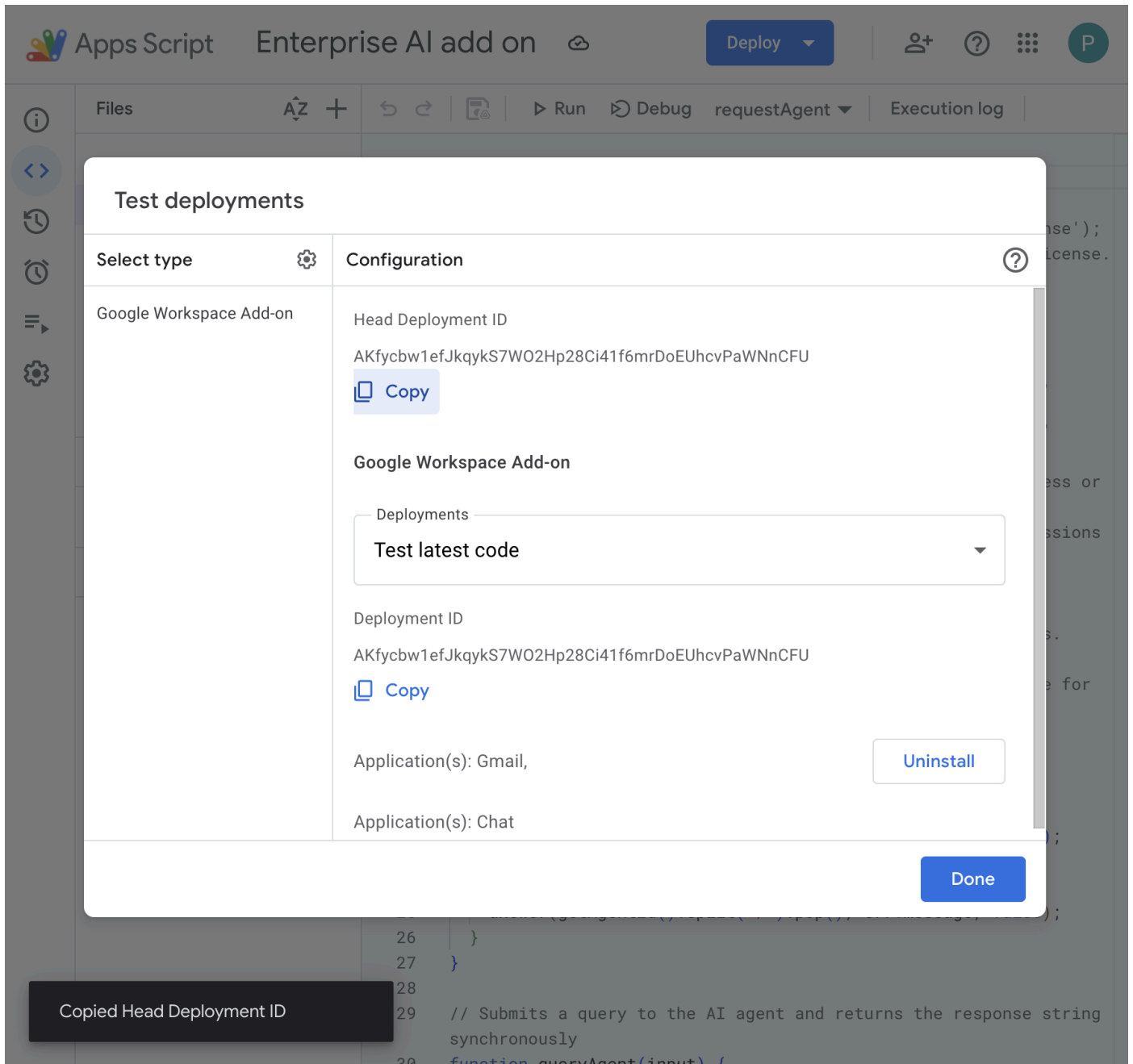
6. 按一下「儲存指令碼屬性」

部署至 Gmail 和 Chat

部署外掛程式，直接在 Gmail 和 Google Chat 中測試。

在 Apps Script 專案中，請按照下列步驟操作：

1. 依序點選「部署」>「測試部署作業」，然後點選「安裝」。這項功能現已在 Gmail 中推出。
2. 按一下「首要部署作業 ID」下方的「複製」。



在 [Google Cloud 控制台](#) 中，按照下列步驟操作：

1. 在 Google Cloud 搜尋欄位中搜尋 `Google Chat API`，依序點選「Google Chat API」、「管理」和「設定」。
2. 選取「啟用互動功能」。
3. 取消選取「加入聊天室和群組對話」。
4. 在「連線設定」下方，選取「Apps Script」。
5. 將「部署作業 ID」設為上一個步驟中複製的「首要部署作業 ID」。
6. 在「瀏覽權限」下方，選取「將這個 Chat 應用程式提供給 Your Workspace Domain 中的特定使用者和群組」，然後輸入電子郵件地址。
7. 按一下 [儲存]。

Google Cloud

ge-gws-agents-lab

Q

☆

API

← API/Service Details

■ Disable API

Configuration

Interactive features

Enable and configure interactive features for a Google Workspace add-on. In Google Chat, add-ons appear to users as Chat apps. [View documentation](#).

☒ Enable Interactive features

Functionality

Users can find and message the app directly in Google Chat. When a user sends the app a direct message, it receives a MESSAGE [event](#).

☐ Join spaces and group conversations

The app can join spaces and group conversations by getting added to them, receiving an ADDED_TO_SPACE [event](#).

Connection settings

Specify a deployment for your Chat app. For guidance, view the [documentation](#).

☐ HTTP endpoint URL

☒ Apps Script

Note: It's recommended to use a versioned deployment for Chat apps in staging or production environments, not a head deployment

☐ Cloud Pub/Sub

☐ Dialogflow

Deployment ID *

AKfycbw1efJkqykS7WO2Hp28Ci41f6mrDoEUhcvPaWNnCFU

I>

Save

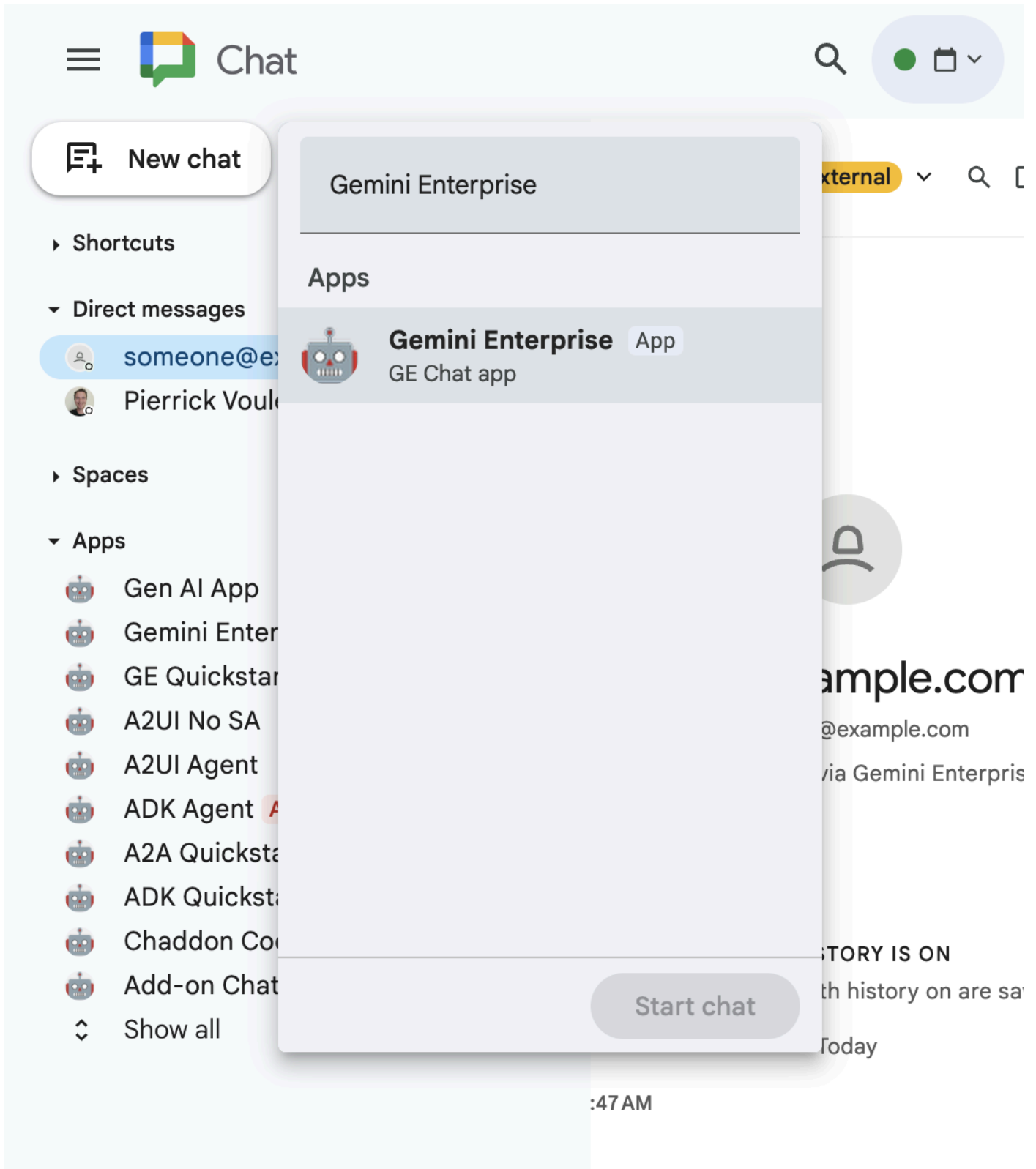
Cancel

試用外掛程式

與即時外掛程式互動，確認外掛程式可以擷取資料，並根據情境回答問題。

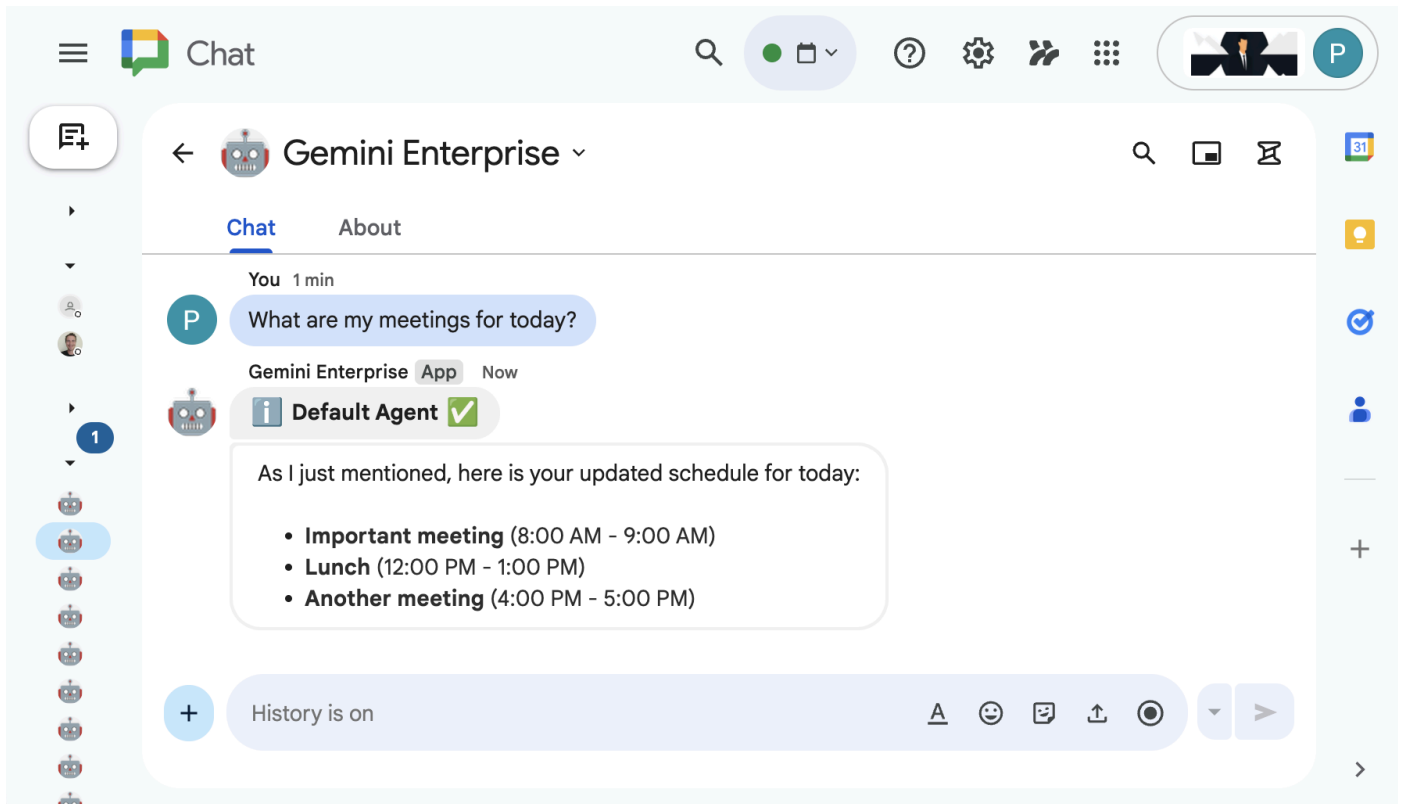
在新分頁中開啟 [Google Chat](#)，然後按照下列步驟操作：

1. 開啟與 Chat 應用程式 **Gemini Enterprise** 互傳的即時訊息。



2. 按一下「設定」，然後完成驗證流程。

3. 輸入 `What are my meetings for today?`，然後按下 `enter` 鍵。**Gemini Enterprise** Chat 應用程式應回覆結果。



在新分頁中開啟 [Gmail](#)，然後按照下列步驟操作：

1. 傳送電子郵件給自己，並將「主旨」設為 `We need to talk`，內文設為 `Are you available today between 8 and 9 AM?`
2. 開啟新收到的電子郵件訊息。
3. 開啟「Enterprise AI」外掛程式側欄。
4. 將「Message」（訊息）設為 `Am I?`
5. 按一下 [傳送訊息]。
6. 答案會顯示在按鈕下方。

A screenshot of the Gmail web interface. The top navigation bar includes the Gmail logo, a search bar, and various utility icons. On the left sidebar, there are buttons for Mail (with a red badge showing 40), Chat, and Meet. The main email view shows an email from 'Pierrick ...' with the subject 'We need to talk' and a timestamp of 'Feb 22, 2026, 10:44 PM (17 hours ago)'. The email body contains the text 'Are you available tomorrow between 8 and 9 AM?'. Below the email, there are buttons for 'Reply', 'Forward', and 'Share in chat' (which has a 'New' badge). An 'Enterprise AI' chat overlay is visible on the right side of the screen, showing a message input field, a 'Send message' button, and the text of the email being discussed: 'No, you are not available tomorrow between 8 and 9 AM. -> You have "DNS" scheduled from 8:00 AM to 10:00 AM.'

6. 清除所用資源

刪除 Google Cloud 專案

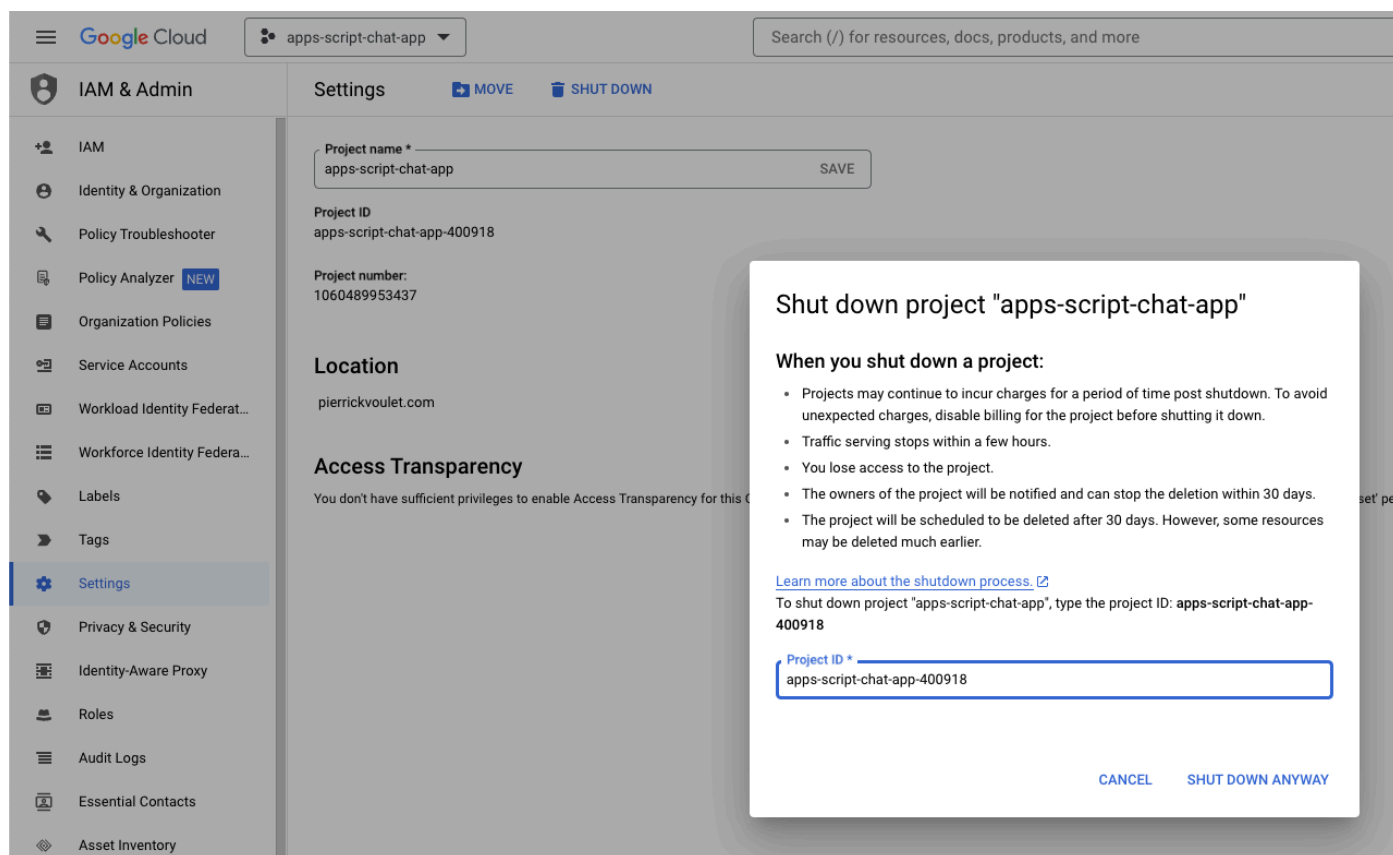
如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，建議您刪除 Google Cloud 專案。

注意：刪除 Google Cloud 專案會導致下列情況：

- 專案中的所有內容都會遭到刪除。如果您使用現有專案進行本程式碼研究室，刪除專案時也會一併刪除您在該專案中執行的其他任何工作。
- 自訂專案 ID 不復存在。您建立這個專案時，可能已建立日後使用的自訂專案 ID。如要保留使用該專案 ID 的網址 (如 appspot.com 上的網址)，請從專案刪除選定的資源，不要刪除整個專案。

在 [Google Cloud 控制台](#) 中，按照下列步驟操作：

- 依序點選「選單」圖示 ≡ > 「IAM 與管理」 > 「設定」。
- 按一下「Shut down」(關閉)。
- 輸入專案 ID。
- 按一下「仍要關閉」。



The screenshot shows the Google Cloud console interface. The left sidebar displays the 'IAM & Admin' section with 'Settings' selected. The main content area shows the 'Settings' page for the project 'apps-script-chat-app'. A modal dialog box titled 'Shut down project "apps-script-chat-app"' is open, displaying a list of consequences when shutting down a project. The dialog includes a text input field for the project ID, which is pre-filled with 'apps-script-chat-app-400918'. At the bottom of the dialog are two buttons: 'CANCEL' and 'SHUT DOWN ANYWAY'.

Shut down project "apps-script-chat-app"

When you shut down a project:

- Projects may continue to incur charges for a period of time post shutdown. To avoid unexpected charges, disable billing for the project before shutting it down.
- Traffic serving stops within a few hours.
- You lose access to the project.
- The owners of the project will be notified and can stop the deletion within 30 days.
- The project will be scheduled to be deleted after 30 days. However, some resources may be deleted much earlier.

[Learn more about the shutdown process.](#)

To shut down project "apps-script-chat-app", type the project ID: **apps-script-chat-app-400918**

Project ID *
apps-script-chat-app-400918

[CANCEL](#) [SHUT DOWN ANYWAY](#)

7. 恭喜

恭喜！您建構的解決方案充分發揮 Gemini Enterprise 和 Google Workspace 的整合效益，讓工作者如虎添翼！

後續步驟

本程式碼研究室只展示最典型的用途，但您可能想在解決方案中考慮許多擴充領域，例如：

- 使用 Gemini CLI 和 Antigravity 等 AI 輔助開發人員工具。
- 與其他代理架構和工具整合，例如自訂 MCP、自訂函式呼叫和生成式 UI。
- 與其他 AI 模型整合，包括在 Agent Platform 等專用平台中託管的自訂模型。
- 與其他代理程式整合，這些代理程式可託管在 Dialogflow 等專用平台，或由第三方透過 Cloud Marketplace 提供。
- 在 Cloud Marketplace 發布代理程式，協助團隊、機構或公開使用者。

瞭解詳情

開發人員可參考許多資源，例如 YouTube 影片、說明文件網站、程式碼範例和教學課程：

- [Google Cloud 開發人員中心](#)
 - [支援的產品 | Google Cloud MCP 伺服器](#)
 - [A2UI](#)
 - [Gemini Enterprise Agent Platform 的 Model Garden | Google Cloud](#)
 - [代理總覽 | Gemini Enterprise | Google Cloud 說明文件](#)
 - [透過 Google Cloud Marketplace 和 Gemini Enterprise 擴大 AI 代理服務規模](#)
 - [透過 Google Cloud Marketplace 提供 AI 虛擬服務專員](#)
 - [Google Workspace 開發人員 YouTube 頻道 - 歡迎開發人員！](#)
 - [Google Workspace 開發人員網站](#)
 - [所有 Google Workspace 外掛程式範例的 GitHub 存放區](#)
-